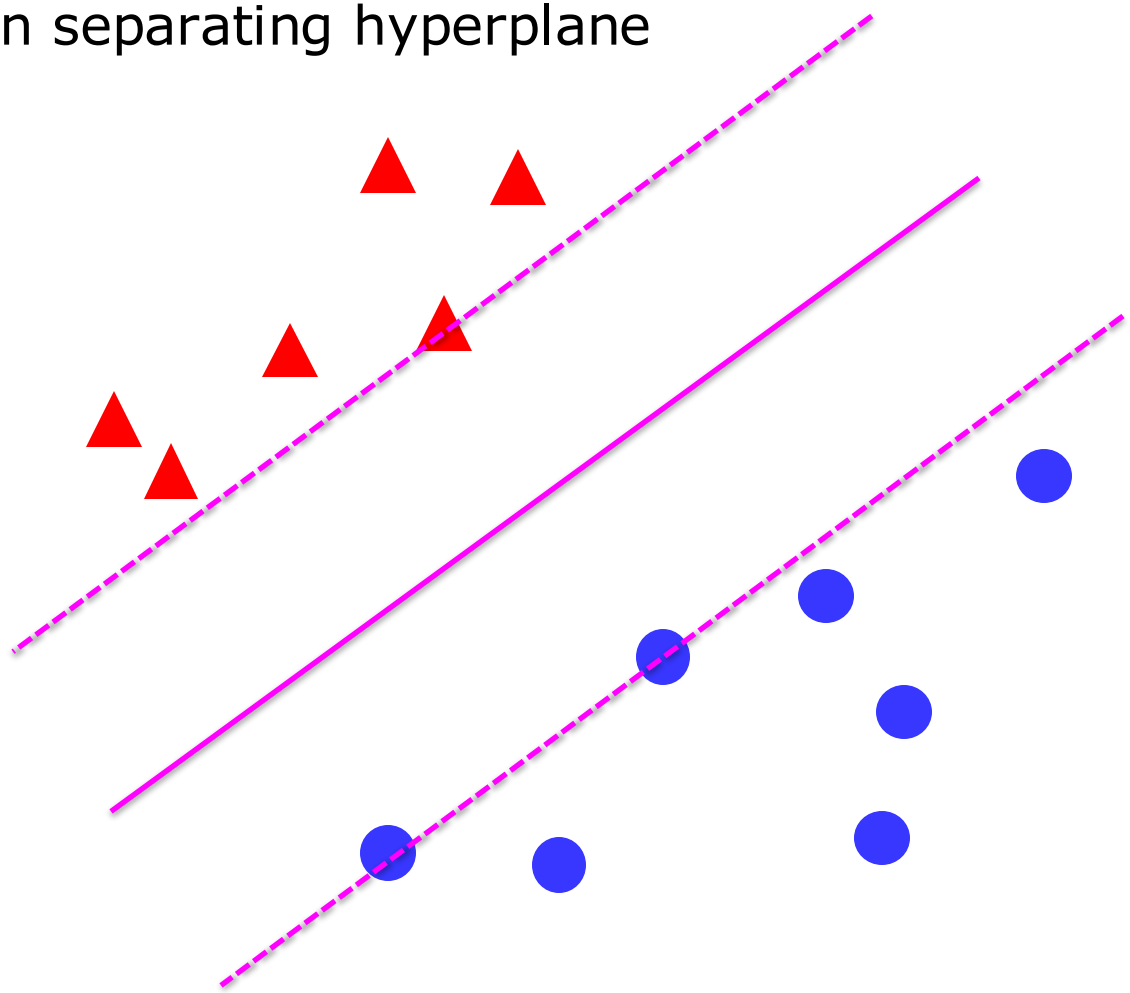
Support Vector Machines

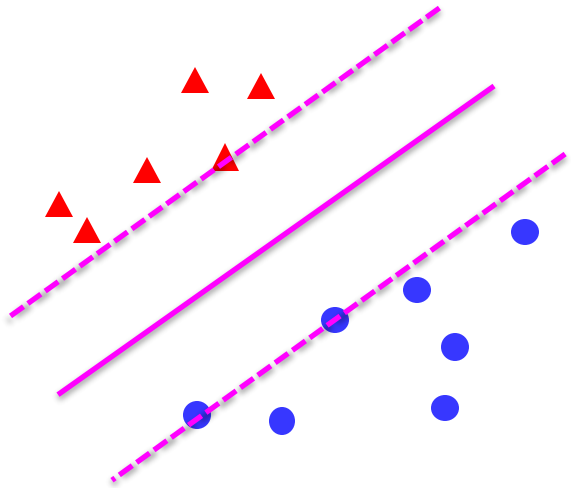
Support Vector Machines

- Max margin separating hyperplane



Optimization Problem via Duality

- SVM optimization problem:
  - $\max_{\alpha:\ \alpha_i \geq 0} \sum_{i=1}^{n} \alpha_i - \frac{1}{2} \sum_{i=1}^{n} \sum_{j=1}^{n} \alpha_i \alpha_j y_i y_j x_i^T x_j$
  - subject to:
    - $\sum_{i=1}^{n} \alpha_i y_i = 0$

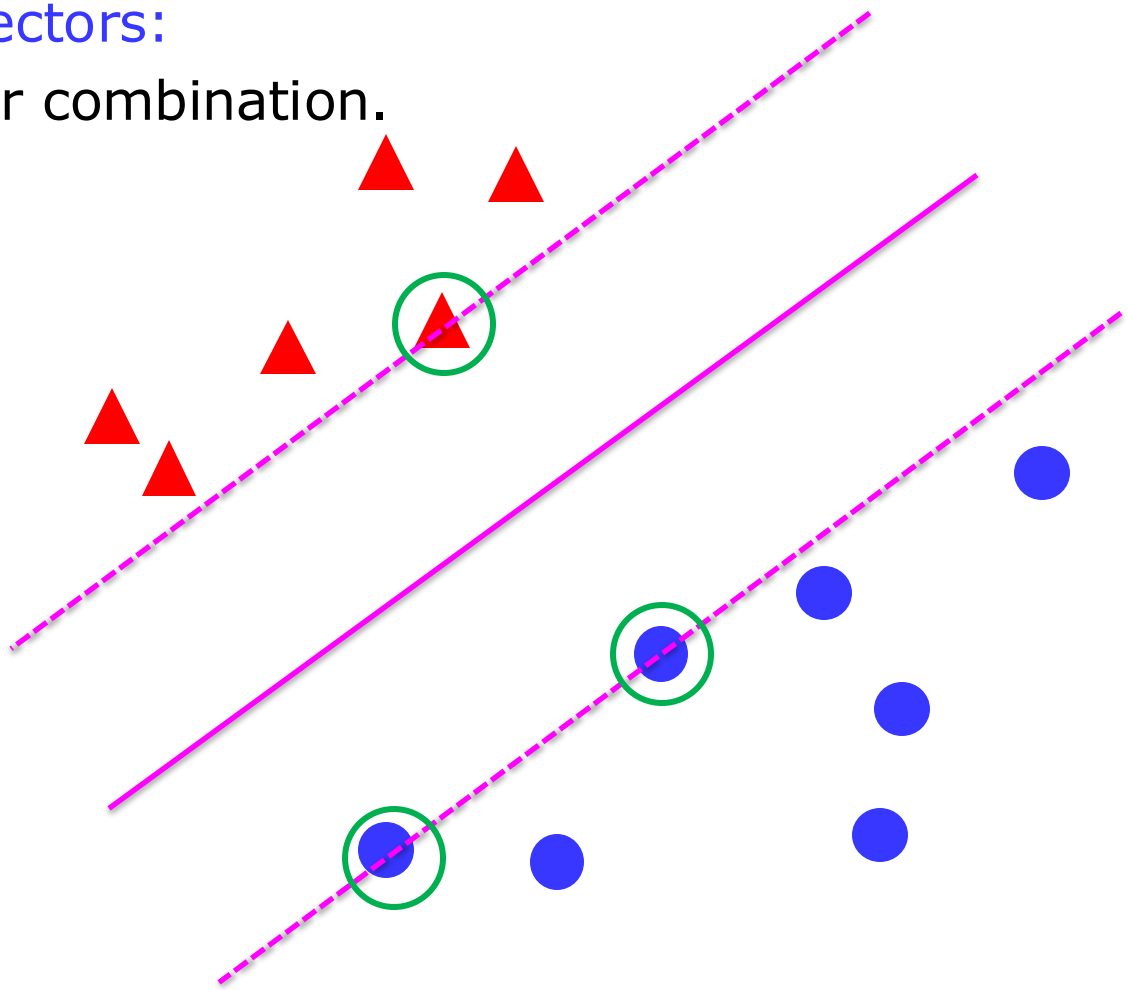


- Once solved, solution hyperplane satisfies:
  - $w = \sum_{i=1}^{n} \alpha_i y_i x_i$
  - $b = -\frac{1}{2}\left(\min_{i:y_i=1} w^T x_i + \max_{i:y_i=-1} w^T x_i\right)$

- $\alpha_i \neq 0$ only for the support vectors.

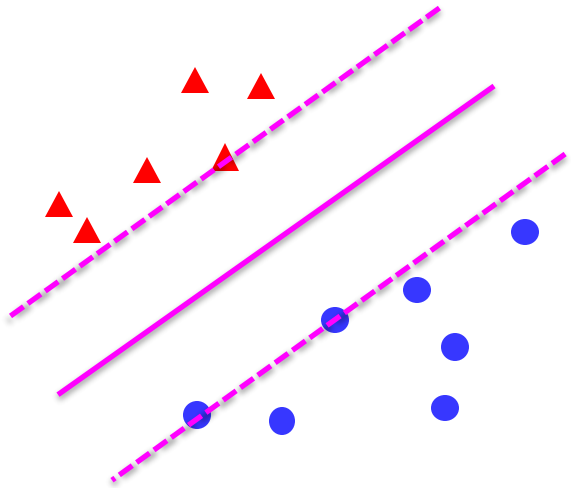
Support Vector Machines

- Support Vectors:
  - w linear combination.



Towards Kernels

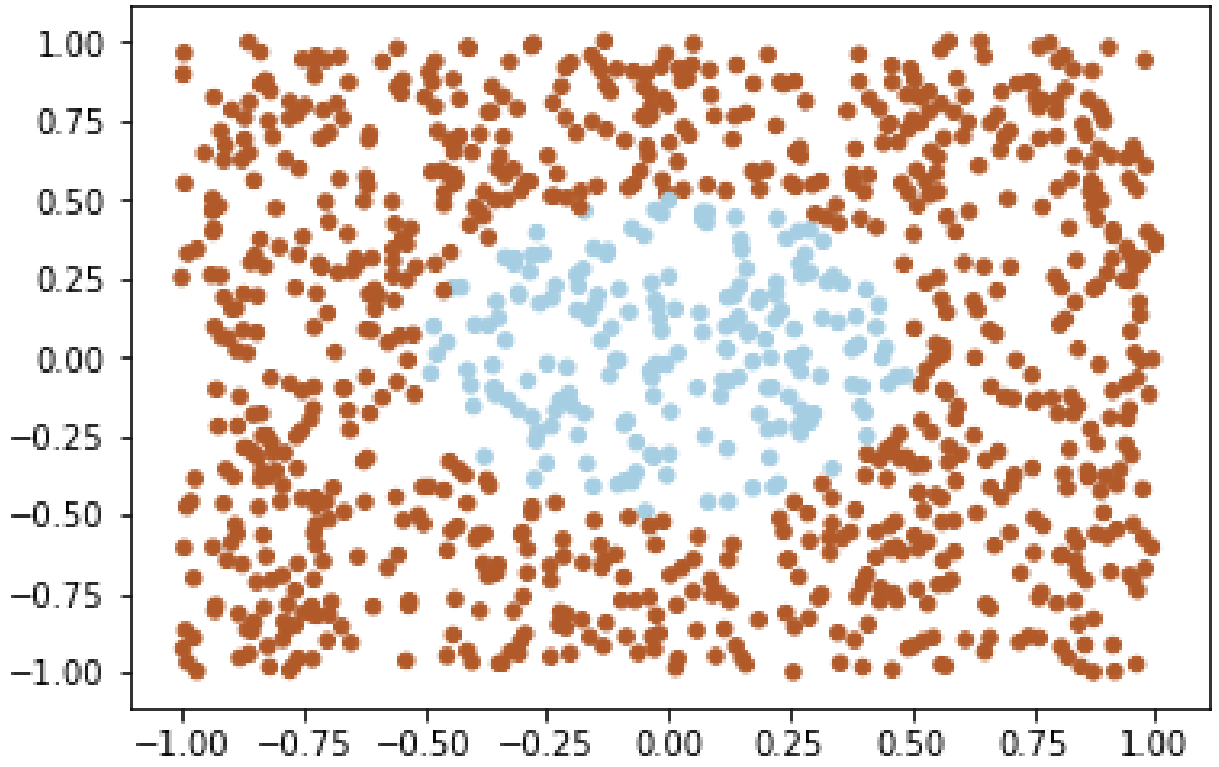
- SVM solution:
  - $w = \sum_{i=1}^{n} \alpha_i y_i x_i$
  - $b = -\frac{1}{2}(\min_{i:y_i=1} w^T x_i + \max_{i:y_i=-1} w^T x_i)$
- $\alpha_i \neq 0$ only for the support vectors.



- Exploiting this for what?
  - Classification of new point x by sign($w^T x+b$).
  - Can also do:
    - $\text{sign}\left(\left(\sum_{\alpha_i \neq 0} \alpha_i y_i x_i\right)^T x + b\right) = \text{sign}\left(\sum_{\alpha_i \neq 0} \alpha_i y_i \, x_i^T x + b\right)$
  - Why would we do that??
    - If k support vectors then time O(kd) compared to O(d) for $w^T x$. Bad idea?!

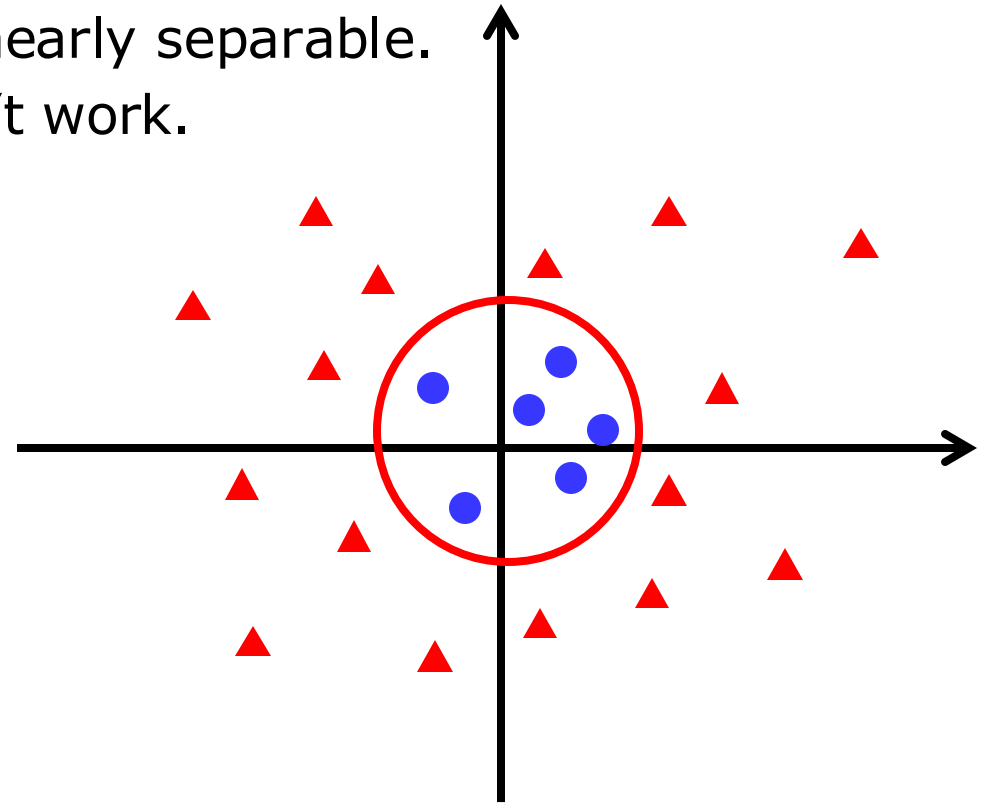
Recap Feature Transforms

- Nonlinear Data:
  - Just abandon linear models?



Recap Feature Transforms

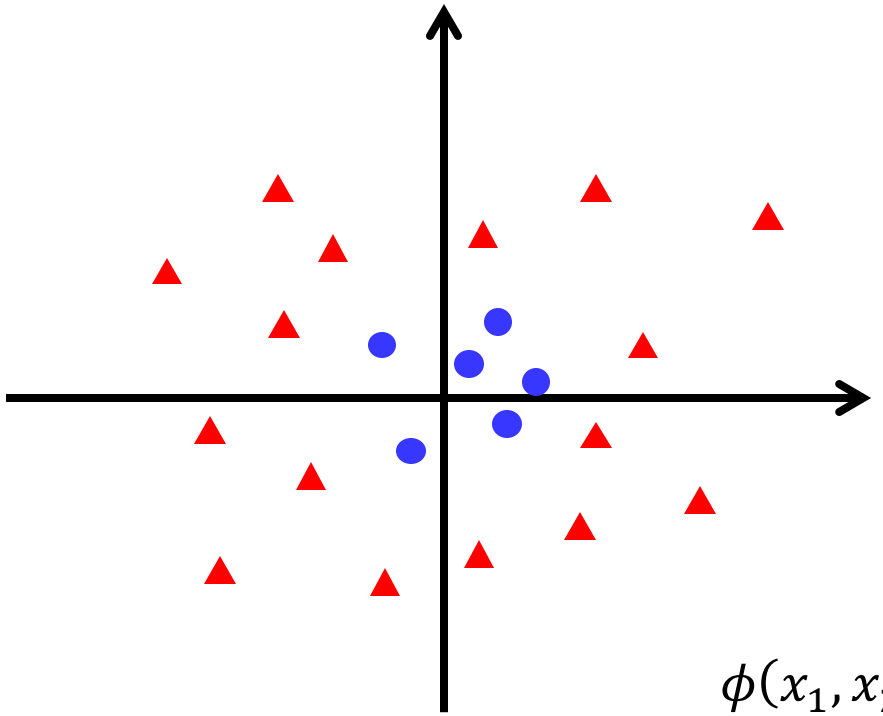
- Example:
  - Far from linearly separable.
  - SVM doesn't work.



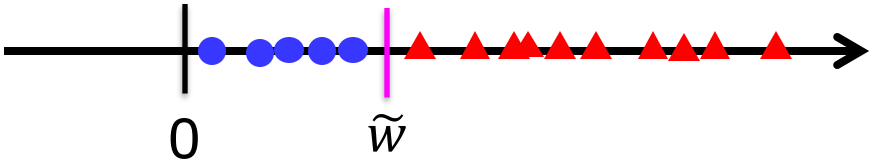
- Looks like distance from origin could classify well.

Recap Feature Transforms

- Nonlinear transforms:
  - Let $\phi : R^d \rightarrow R^{d'}$ be any mapping from R^d to $R^{d'}$.
  - Apply ϕ to all feature vectors before training SVM.
  - Get new data matrix X' which is n x d' instead of n x d.



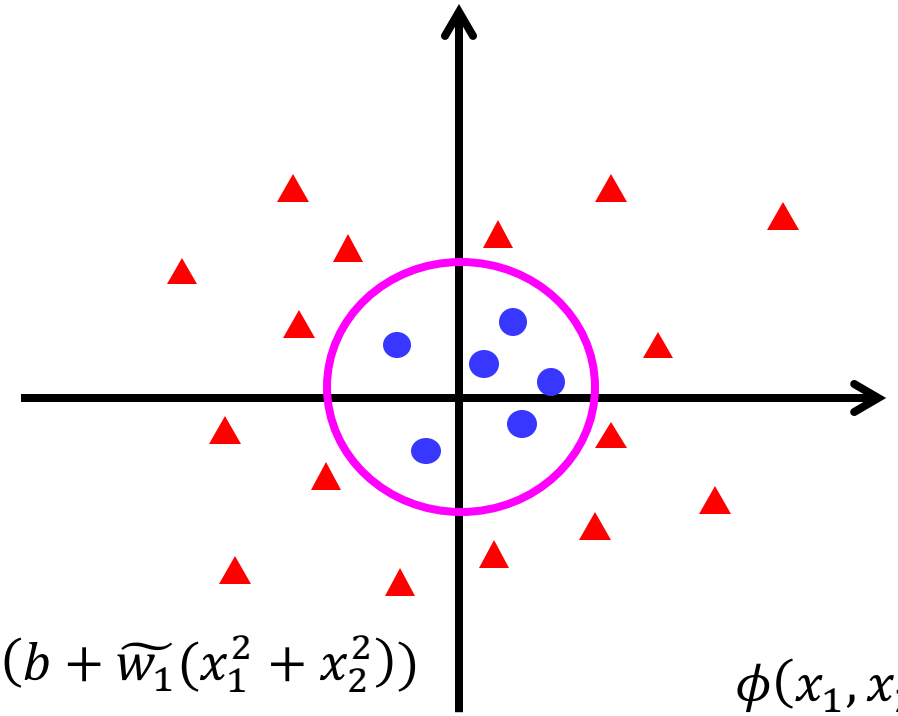
Run SVM on X' and find separating hyperplane $\tilde{w}^T x + b$.



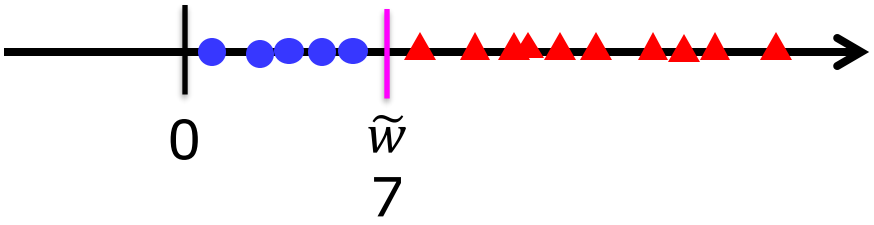
$$\phi(x_1, x_2) = (x_1^2 + x_2^2)$$

Recap Feature Transforms

- Effective hypothesis: $\text{sign}(\widetilde{w}^T \phi(x_1, x_2) + b)$



Run SVM on X' and find separating hyperplane $\widetilde{w}^T x + b$.



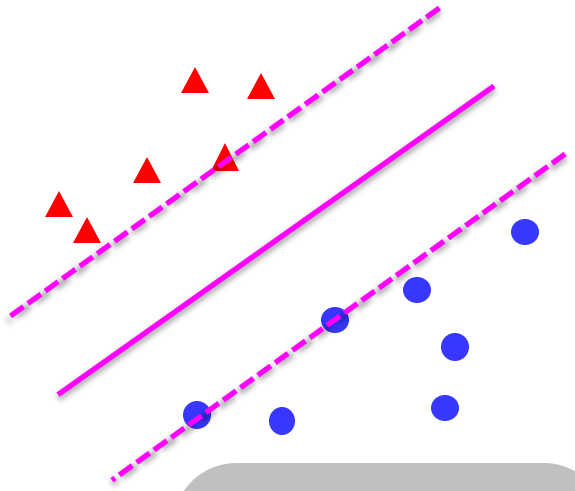
$\text{sign}(b + \widetilde{w_1}(x_1^2 + x_2^2))$

$\text{sign}(7 - 1(x_1^2 + x_2^2))$

$\phi(x_1, x_2) = (x_1^2 + x_2^2)$

Towards Kernels

- SVM solution:
  - $w = \sum_{i=1}^{n} \alpha_i y_i x_i$
  - $b = -\frac{1}{2}(\min_{i:y_i=1} w^T x_i + \max_{i:y_i=-1} w^T x_i)$
- $\alpha_i \neq 0$ only for the support vectors.

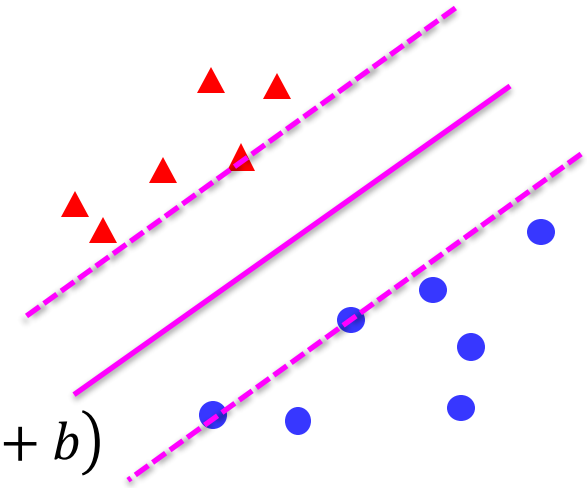


- SVM and feature transform ϕ: $R^d \rightarrow R^{d'}$
  - $w = \sum_{i=1}^{n} \alpha_i y_i \phi(x_i)$
  - $b = -\frac{1}{2}(\min_{i:y_i=1} w^T \phi(x_i) + \max_{i:y_i=-1} w^T \phi(x_i))$
- $\alpha_i \neq 0$ only for the support vectors (in feature spac
- Prediction:
  - sign($w^T \phi(x)$+b) or $\text{sign}\left(\sum_{\alpha_i \neq 0} \alpha_i y_i \, \phi(x_i)^T \phi(x) + b\right)$

O(d') vs O(kd'). Still not clear why do this.

Towards Kernels

- SVM and feature transform ϕ: $R^d \rightarrow R^{d'}$
  - $w = \sum_{i=1}^{n} \alpha_i y_i \phi(x_i)$
  - $b = -\frac{1}{2}(\min_{i:y_i=1} w^T \phi(x_i) + \max_{i:y_i=-1} w^T \phi(x_i))$
- $\alpha_i \neq 0$ only for the support vectors.
- Prediction:
  - sign($w^T \phi(x)$+b) or $\text{sign}\left(\sum_{\alpha_i \neq 0} \alpha_i y_i \, \phi(x_i)^T \phi(x) + b\right)$



- Key Observation:
  - Second method only depends on inner products of feature transformed vectors. First method needs transformed vectors explicitly.
  - How can we exploit this?

Kernels

- Feature transform ϕ: $R^d \rightarrow R^{d'}$
  - Corresponding kernel K: R^d x $R^d \rightarrow R$
  - $K(x,z) = \phi(x)^T \phi(z)$.

- Rewriting SVM in terms of kernel:
  - $\max_{\alpha:\, \alpha_i \geq 0} \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j y_i y_j x_i^T x_j$
  - subject to:
    - $\sum_{i=1}^n \alpha_i y_i = 0$
- Becomes:
  - $\max_{\alpha:\, \alpha_i \geq 0} \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j y_i y_j K(x_i, x_j)$
  - subject to:
    - $\sum_{i=1}^n \alpha_i y_i = 0$

Kernels

- Feature transform ϕ: $R^d \rightarrow R^{d'}$
  - Corresponding kernel K: R^d x $R^d \rightarrow R$
  - $K(x,z) = \phi(x)^T \phi(z)$.

- Kernel SVM formulation:
  - $\max_{\alpha:\, \alpha_i \geq 0} \sum_{i=1}^{n} \alpha_i - \frac{1}{2} \sum_{i=1}^{n} \sum_{j=1}^{n} \alpha_i \alpha_j y_i y_j K(x_i, x_j)$
  - subject to:
    - $\sum_{i=1}^{n} \alpha_i y_i = 0$
- Bias:
  - $b = -\frac{1}{2} (\min_{i:y_i=1} w^T x_i + \max_{i:y_i=-1} w^T x_i)$
  - $b = -\frac{1}{2} (\min_{i:y_i=1} \sum_{\alpha_j \neq 0} \alpha_j y_j K(x_i, x_j) + \max_{i:y_i=-1} \sum_{\alpha_j \neq 0} \alpha_j y_j K(x_i, x_j))$

Kernels

- Feature transform ϕ: $R^d \rightarrow R^{d'}$
  - Corresponding kernel K: R^d x $R^d \rightarrow R$
  - $K(x,z) = \phi(x)^T \phi(z)$.

Entire algorithm can be written in terms of kernel. Never need feature transformed vectors explicitly.

- Kernel SVM formulation:
  - $\max_{\alpha:\, \alpha_i \geq 0} \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j y_i y_j K(x_i, x_j)$
  - subject to:
    - $\sum_{i=1}^n \alpha_i y_i = 0$
  - $b = -\frac{1}{2} \left(\min_{i: y_i = 1} \sum_{\alpha_j \neq 0} \alpha_j y_j K(x_i, x_j) + \max_{i: y_i = -1} \sum_{\alpha_j \neq 0} \alpha_j y_j K(x_i, x_j) \right)$
- Prediction on x:
  - $\text{sign}\left(\sum_{\alpha_i \neq 0} \alpha_i y_i K(x_i, x) + b \right)$

Finds maximum margin separating hyperplane in feature space!

Kernels

- Feature transform ϕ: $R^d \rightarrow R^{d'}$
  - Corresponding kernel K: R^d x $R^d \rightarrow R$
  - $K(x,z) = \phi(x)^T \phi(z)$.

- Evaluating the kernel:
  - Direct approach: Compute $\phi(x)$ and $\phi(z)$ and then compute inner product.
    - Time $\Omega(d')$ (at least).
  - Didn't gain anything over working directly in feature space.
  - Some kernels are faster to evaluate than to perform transform first!

Kernels

- Feature transform ϕ: $R^d \rightarrow R^{d'}$
  - Corresponding kernel K: R^d x $R^d \rightarrow R$
  - $K(x,z) = \phi(x)^T \phi(z)$.

- Polynomial kernel:
  - $K(x,z) = (x^T z)^2$
  - Can compute in O(d) time!
  - Wait! Does this even correspond to $\phi(x)^T \phi(z)$ for some feature transform?

Kernels

- Feature transform ϕ: $R^d \to R^{d'}$
  - Corresponding kernel K: R^d x $R^d \to R$
  - $K(x,z) = \phi(x)^T \phi(z)$.

$$\phi(x) = \begin{pmatrix} x_1 x_1 \\ x_1 x_2 \\ \vdots \\ x_d x_{d-1} \\ x_d x_d \end{pmatrix}$$

- Polynomial kernel:
  - $K(x,z) = (x^T z)^2$
  - Can compute in O(d) time!

  - $K(x,z) = (x^T z)^2 = \left(\sum_{i=1}^d x_i z_i\right)\left(\sum_{j=1}^d x_j z_j\right) = \sum_{i=1}^d \sum_{j=1}^d x_i z_i x_j z_j =$
  - $\sum_{i=1}^d \sum_{j=1}^d (x_i x_j)(z_i z_j)$
  - $= \phi(x)^T \phi(z)$

Kernels

- Feature transform ϕ: $R^d \rightarrow R^{d'}$
  - Corresponding kernel K: R^d x $R^d \rightarrow R$
  - $K(x,z) = \phi(x)^T \phi(z)$.

$$\phi(x) = \begin{pmatrix} x_1 x_1 \\ x_1 x_2 \\ \vdots \\ x_d x_{d-1} \\ x_d x_d \end{pmatrix}$$

- Polynomial kernel:
  - $K(x,z) = (x^T z)^2$
  - Can compute in O(d) time!
  - Corresponds to feature transform ϕ into $d'=d^2$.
    - Performing transform would take $\Omega(d^2)$ time.

Kernels

- Feature transform ϕ: $R^d \to R^{d'}$
  - Corresponding kernel K: R^d x $R^d \to R$
  - $K(x,z) = \phi(x)^T \phi(z)$.

$$\phi(x) = \begin{pmatrix} x_1 x_1 \cdots x_1 \\ x_1 x_1 \cdots x_2 \\ \vdots \\ x_d x_d \cdots x_{d-1} \\ x_d x_d \cdots x_d \end{pmatrix}$$

- Polynomial kernel:
  - $K(x,z) = (x^T z)^r$
  - Can compute in O(d) time!

  - $K(x,z) = (x^T z)^r = \sum_{i_1 i_2,\ldots,i_r} x_{i_1} z_{i_1} \cdots x_{i_r} z_{i_r} = \sum_{i_1 i_2,\ldots,i_r} (x_{i_1} \cdots x_{i_r})(z_{i_1} \cdots z_{i_r})$

  - Corresponds to a feature transform into d^r dimensions!
  - O(d) time instead of $O(d^r)$!

Kernels

- Feature transform ϕ: $R^d \to R^{d'}$
  - Corresponding kernel K: R^d x $R^d \to R$
  - $K(x,z) = \phi(x)^T \phi(z)$.

$$\phi(x) = \begin{pmatrix} c \\ \sqrt{2c}x_1 \\ \vdots \\ \sqrt{2c}x_d \\ x_1x_1 \\ x_1x_2 \\ \vdots \\ x_dx_{d-1} \\ x_dx_d \end{pmatrix}$$

- Polynomial kernel:
  - $K(x,z) = (x^Tz+c)^2$ for hardcoded constant c.
  - Can compute in O(d) time!

  - $K(x,z) = (x^Tz+c)^2 = \left(c + \sum_{i=1}^d x_iz_i\right)\left(c + \sum_{j=1}^d x_jz_j\right)$
  - $= cc + 2c\sum_{i=1}^d x_iz_i + \sum_{i=1}^d \sum_{j=1}^d (x_ix_j)(z_iz_j)$

- c gives a way to weigh degree 1 vs degree 2 terms.

Kernels

- Feature transform ϕ: $R^d \rightarrow R^{d'}$
  - Corresponding kernel K: R^d x $R^d \rightarrow R$
  - $K(x,z) = \phi(x)^T \phi(z)$.

- Polynomial kernel:
  - $K(x,z) = (x^T z + c)^r$ for hardcoded constant c.
  - Can compute in O(d) time!

- c gives a way to weigh terms (degree j weighed by $\sqrt{c^{r-j}}$)

Kernels

- Kernel SVM formulation:
  - $\max_{\alpha:\ \alpha_i \geq 0} \sum_{i=1}^{n} \alpha_i - \frac{1}{2} \sum_{i=1}^{n} \sum_{j=1}^{n} \alpha_i \alpha_j y_i y_j K(x_i, x_j)$
  - subject to:
    - $\sum_{i=1}^{n} \alpha_i y_i = 0$
  - $b = -\frac{1}{2} \left(\min_{i: y_i = 1} \sum_{\alpha_j \neq 0} \alpha_j y_j K(x_i, x_j) + \max_{i: y_i = -1} \sum_{\alpha_j \neq 0} \alpha_j y_j K(x_i, x_j) \right)$
- Prediction on x:
  - $\text{sign}\left(\sum_{\alpha_i \neq 0} \alpha_i y_i K(x_i, x) + b \right)$
- Running time: Assume K takes t time to evaluate.
  - Assume k support vectors.
  - $O(n^2 t)$ + normal SVM time to solve. Predict in $O(kt)$.

$O(n^2 t)$ to prepare.

$O(knt)$

$O(kt)$

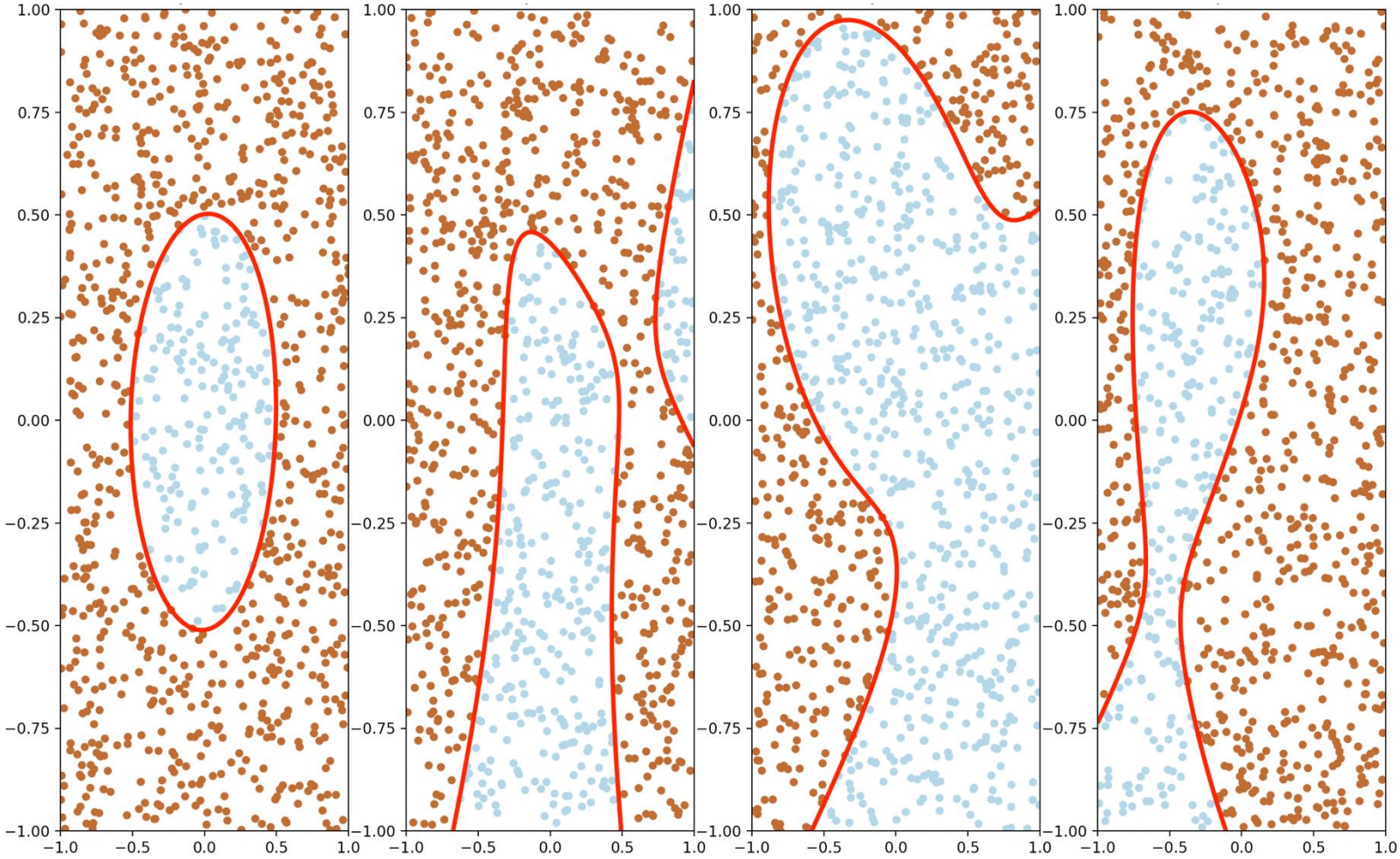
Kernels

- Running time: Assume K takes t time to evaluate.
  - Assume k support vectors.
  - $O(n^2t)$ + normal SVM time to solve. Predict in O(kt).

- Running time if did feature transform explicitly:
  - $O(n^2d')$ + normal SVM time to solve. Predict in O(d').

- Prediction time O(kt) vs O(d'). Which is better? Depends on number of support vectors k.

Kernels



Gaussian Kernel

- Polynomial kernel:
  - $K(x,z) = (x^T z+c)^r$ for hardcoded constant c.

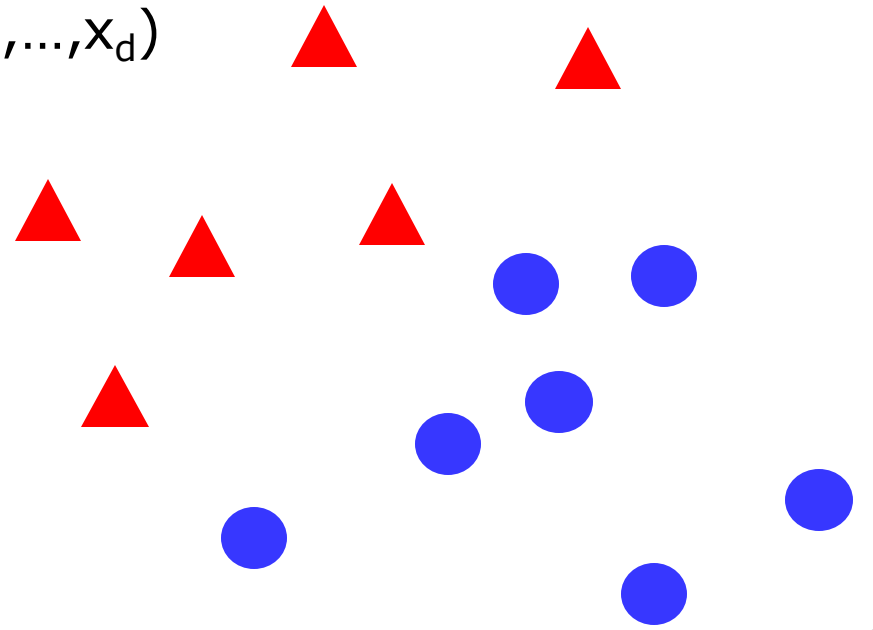
- Gaussian kernel:
  - $K(x,z) = \exp(-\frac{\|x-z\|^2}{2\sigma^2})$ for hardcoded constant $\sigma > 0$.
  - Inspired by probability density function of Gaussian/normal distribution.
  - Large inner products if x and z are close. Similar to "normal" inner products.
  - Simpler to state as:
    - $K(x,z) = \exp(-\lambda\|x-z\|^2)$ for hardcoded constant $\lambda > 0$.
  - Can be shown to correspond to infinite-dimensional feature transform!

Kernels

- Kernel SVM formulation:
  - $\max_{\alpha:\ \alpha_i \geq 0} \sum_{i=1}^{n} \alpha_i - ½ \sum_{i=1}^{n} \sum_{j=1}^{n} \alpha_i \alpha_j y_i y_j K(x_i, x_j)$
  - subject to:
    - $\sum_{i=1}^{n} \alpha_i y_i = 0$
  - $b = -\frac{1}{2}\left(\min_{i:y_i=1} \sum_{\alpha_j \neq 0} \alpha_j y_j\ K(x_i, x_j) + \max_{i:y_i=-1} \sum_{\alpha_j \neq 0} \alpha_j y_j\ K(x_i, x_j)\right)$
- Prediction on x:
  - $\text{sign}\left(\sum_{\alpha_i \neq 0} \alpha_i y_i\ K(x_i, x) + b\right)$
- Not restricted to Support Vector Machines!

Perceptron Learning Algorithm (PLA)

- Surprisingly simple algorithm!
  - Features: $x=(x_0,x_1,\ldots,x_d)$
  - Labels: {-1,1}



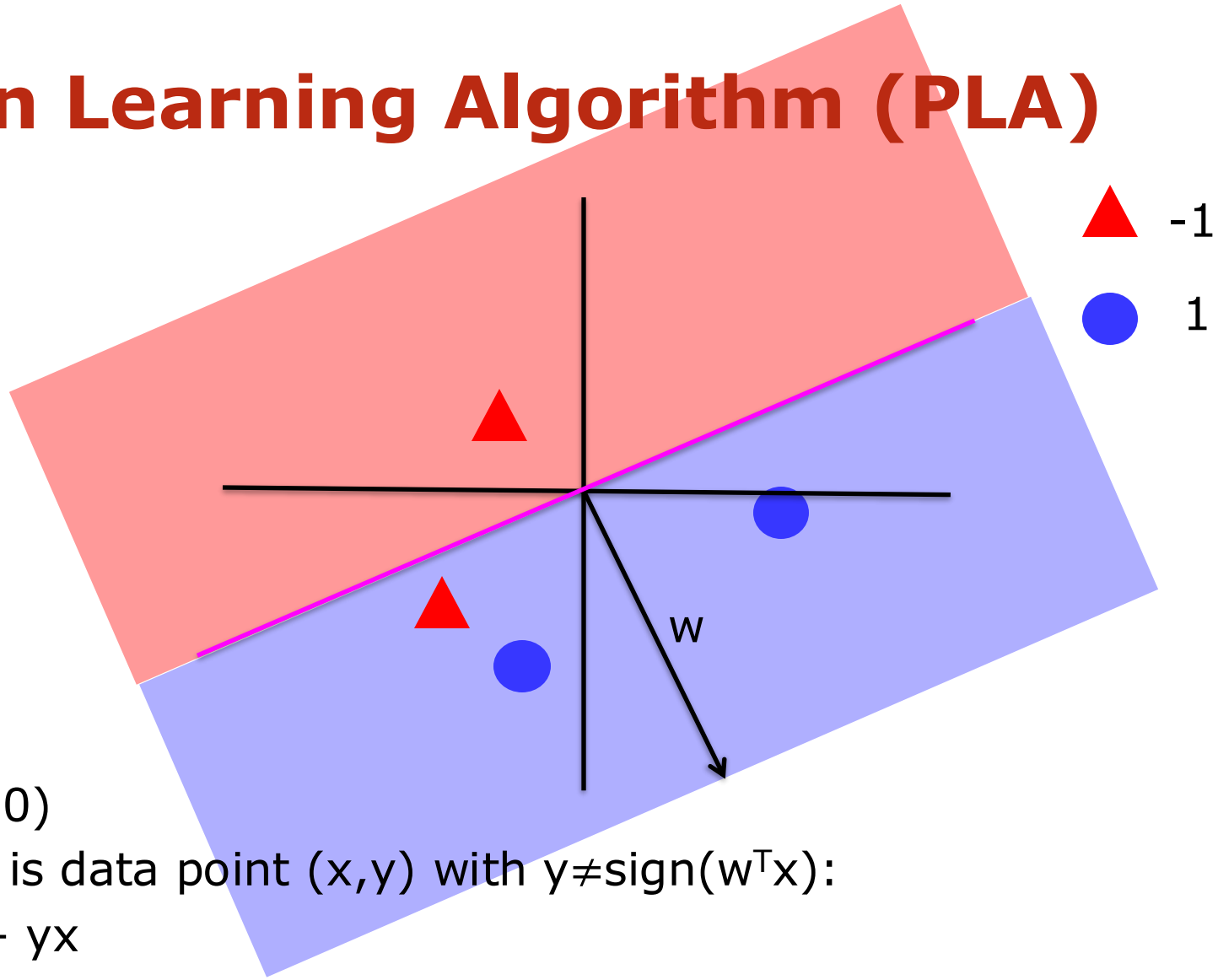
- Algorithm:
  - w = (0,0,…,0)
  - While there is data point (x,y) with $y \neq \text{sign}(w^T x)$:
    - w = w + yx
  - Return w

Wrong classification

Make w "more" correct on (x,y)

Perceptron Learning Algorithm (PLA)

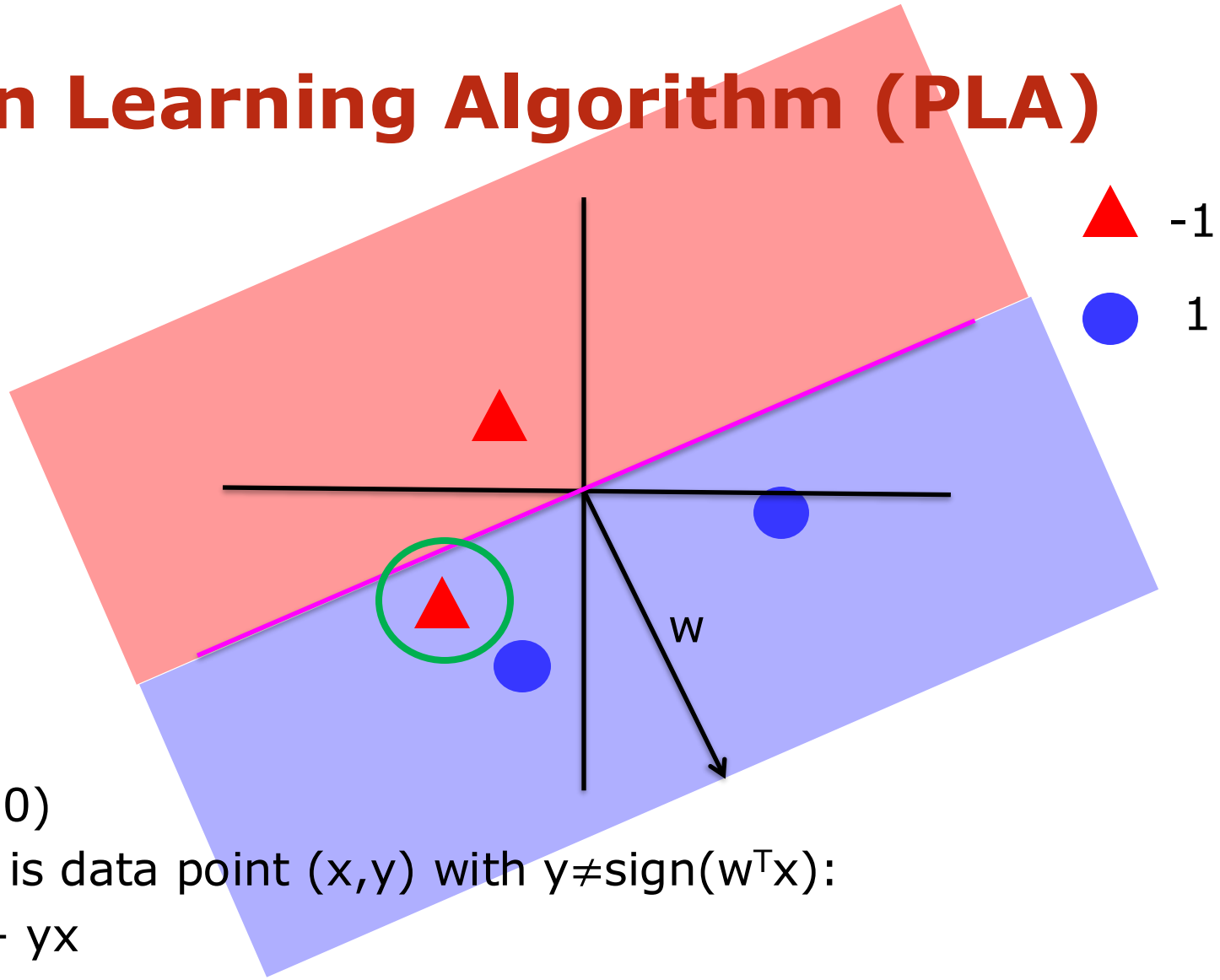
- Example:



- Algorithm:
  - w = (0,0,…,0)
  - While there is data point (x,y) with $y \neq \text{sign}(w^T x)$:
    - w = w + yx
  - Return w

Perceptron Learning Algorithm (PLA)

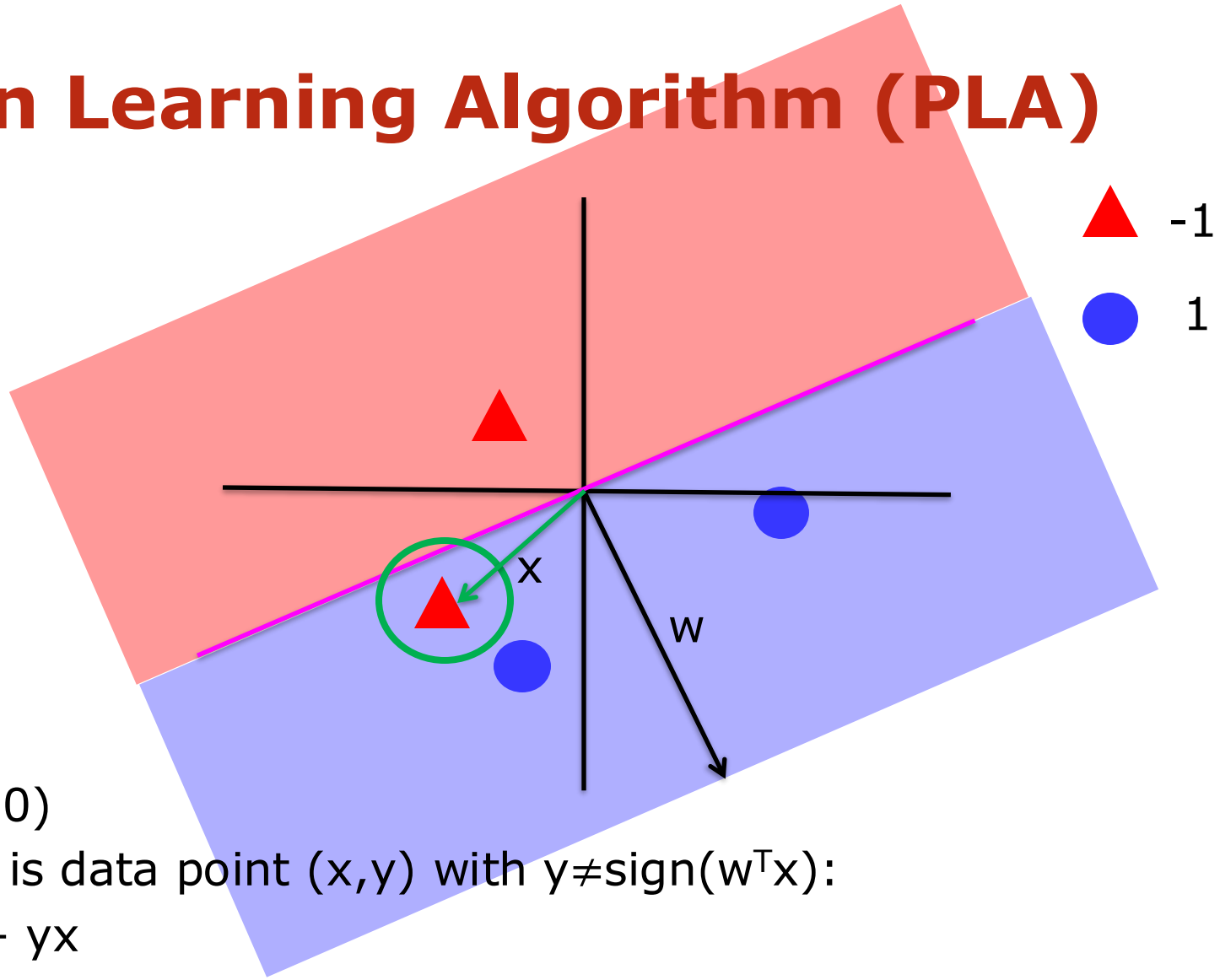
- Example:



- Algorithm:
  - w = (0,0,…,0)
  - While there is data point (x,y) with $y \neq \text{sign}(w^T x)$:
    - w = w + yx
  - Return w

Perceptron Learning Algorithm (PLA)

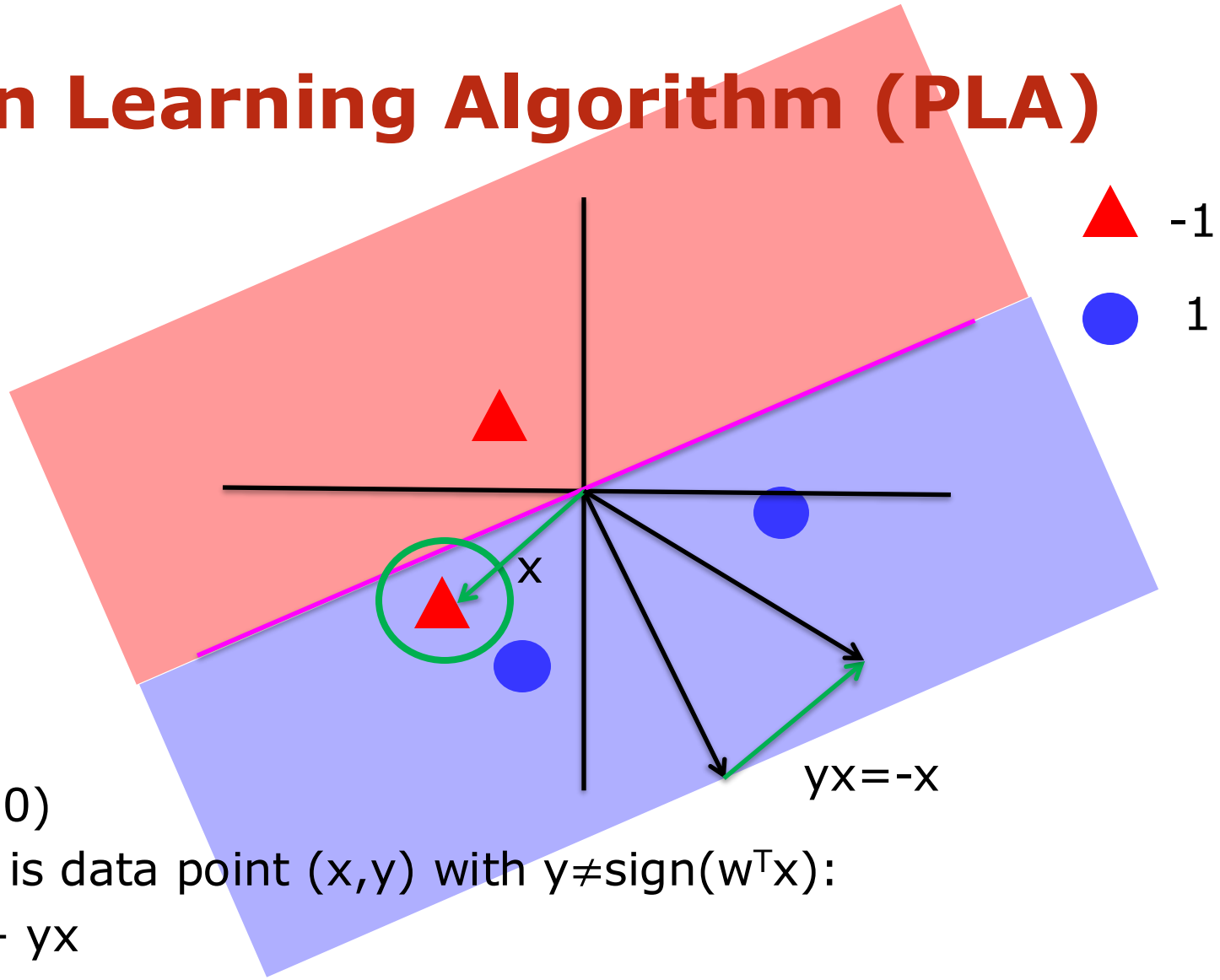
- Example:



- Algorithm:
  - w = (0,0,…,0)
  - While there is data point (x,y) with $y \neq \text{sign}(w^T x)$:
    - w = w + yx
  - Return w

Perceptron Learning Algorithm (PLA)

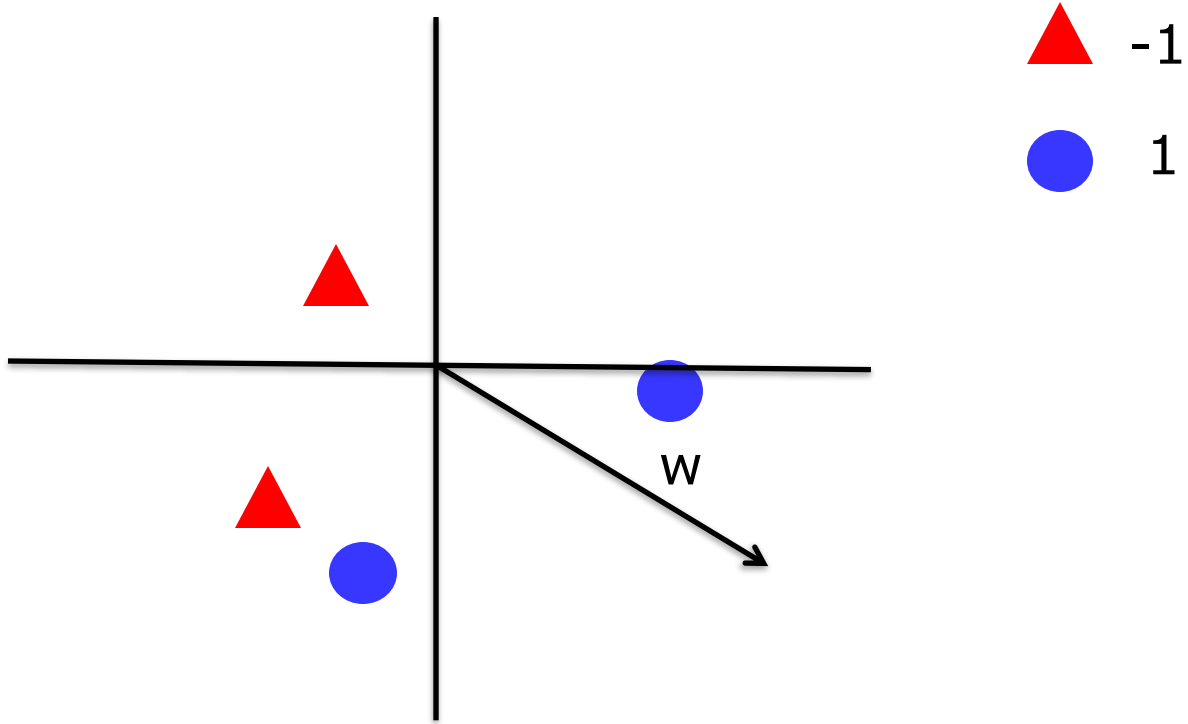
- Example:



- Algorithm:
  - w = (0,0,…,0)
  - While there is data point (x,y) with $y \neq \text{sign}(w^T x)$:
    - w = w + yx
  - Return w

Perceptron Learning Algorithm (PLA)

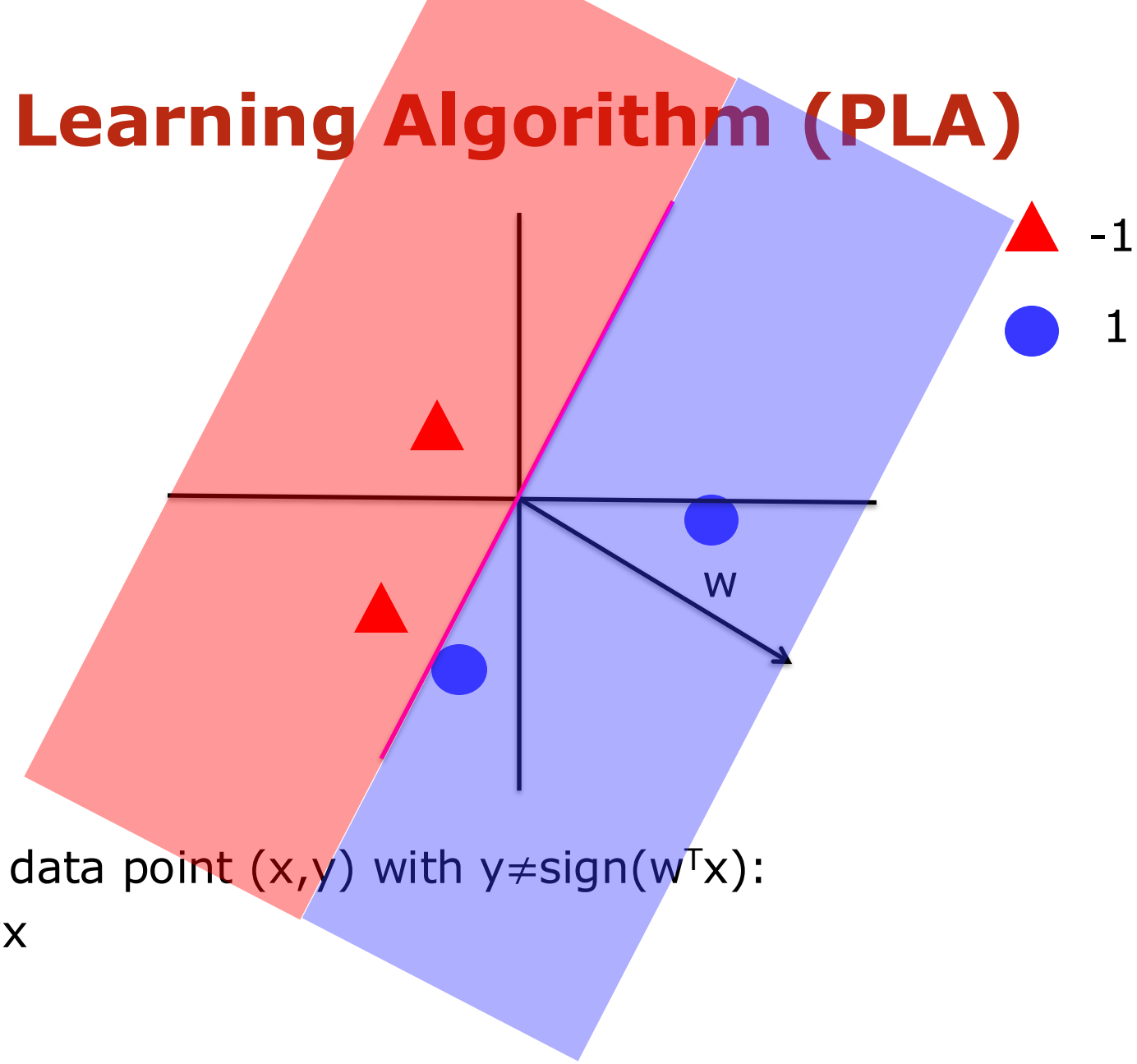
- Example:



- Algorithm:
  - w = (0,0,…,0)
  - While there is data point (x,y) with $y \neq \text{sign}(w^T x)$:
    - w = w + yx
  - Return w

Perceptron Learning Algorithm (PLA)

- Example:



- Algorithm:
  - w = (0,0,…,0)
  - While there is data point (x,y) with $y \neq \text{sign}(w^T x)$:
    - w = w + yx
  - Return w

PLA with Feature Transform

- With feature transform ϕ

- Algorithm:
  - w = (0,0,…,0)
  - While there is data point (x,y) with $y \neq \text{sign}(w^T \phi(x))$:
    - $w = w + y\phi(x)$
  - Return w

- Observation:
  - w is just a sum of feature transformed input vectors.
  - Only compute inner products against w.

PLA with Feature Transform

- With feature transform ϕ

- Algorithm:
  - wList=[]
  - aList=[]
  - While there is data point (x,y) with $y \neq \text{sign}((\sum_{i=1}^{|\text{wList}|} \text{aList}[i] \cdot \text{wList}[i])^T \phi(x))$:
    - wList.append(ϕ(x))
    - aList.append(y)
  - Return (wList,aList)

PLA with Feature Transform

- With feature transform ϕ

- Algorithm:
  - wList=[]
  - aList=[]
  - While there is data point (x,y) with $y \neq \text{sign}(\sum_{i=1}^{|\text{wList}|} \text{aList}[i] \cdot (\text{wList}[i]^T \phi(x)))$:
    - wList.append(ϕ(x))
    - aList.append(y)
  - Return (wList,aList)

PLA with Kernel

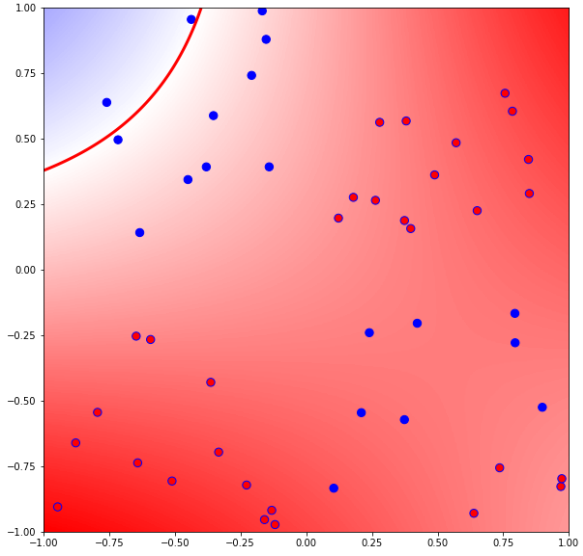
- With kernel K

- Algorithm:
  - wList=[]
  - aList=[]
  - While there is data point (x,y) with $y \neq \text{sign}(\sum_{i=1}^{|\text{wList}|} \text{aList}[i] \cdot \text{K}(\text{wList}[i], x))$:
    - wList.append(x)
    - aList.append(y)
  - Return (wList,aList)

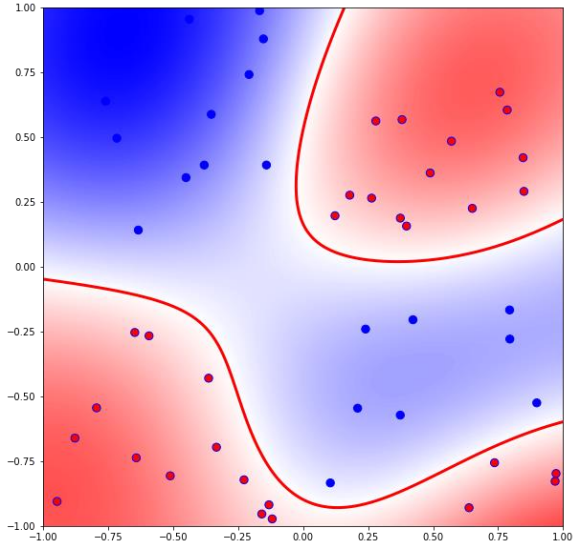
- Predict: $\text{sign}(\sum_{i=1}^{|\text{wList}|} \text{aList}[i] \cdot \text{K}(\text{wList}[i], x))$

PLA with Kernel

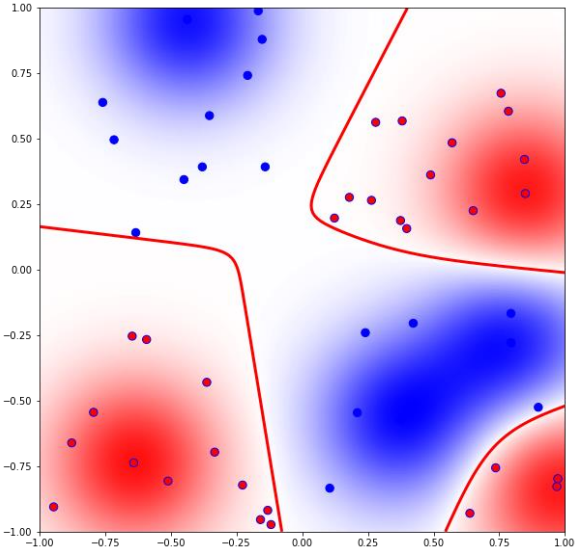
- PLA with Gaussian kernel
  - K(x,z) = $\exp(-\lambda\|x - z\|^2)$



$\lambda = 0.1$



$\lambda = 1$



$\lambda = 10$

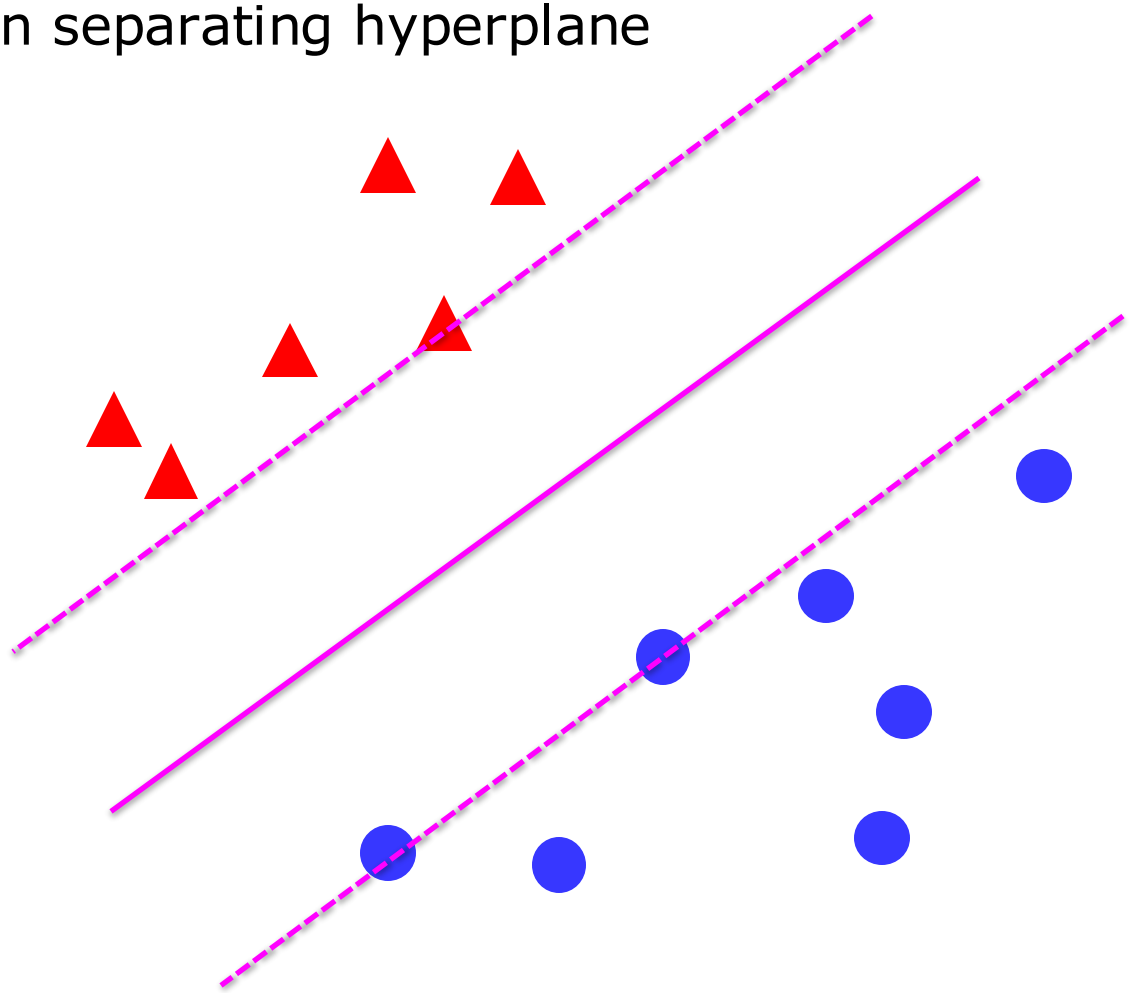
Some Considerations

- Pitfalls:
  - We saw in learning theory that the complexity of the hypothesis set H matters for generalization.
    - Low complexity better.
  - So, should try to avoid too many features after transform.
    - E.g. might be bad to use polynomial kernel $(x^T z+c)^r$ with too large r.
    - Gaussian kernel maps to infinite dimensions, which seems very bad.
    - There is theory that can explain why often still works well even in infinite dimensions.
- Choosing c and λ:
  - As always, use validation set and try multiple values.

Support Vector Machines

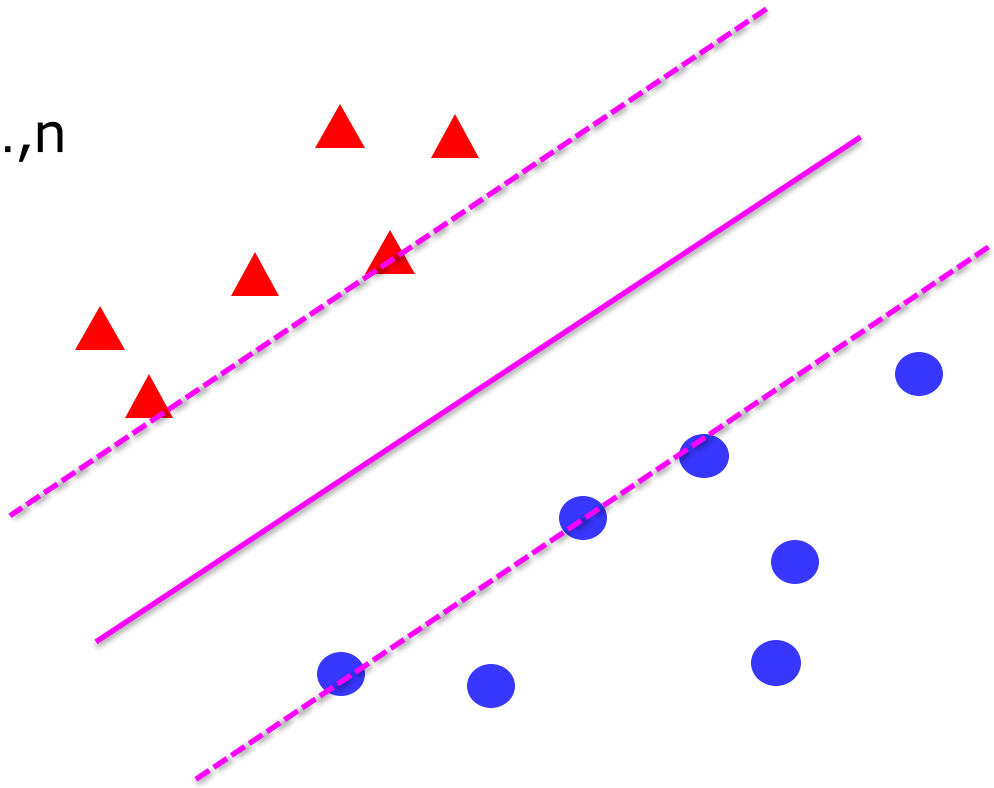
Support Vector Machines

- Max margin separating hyperplane



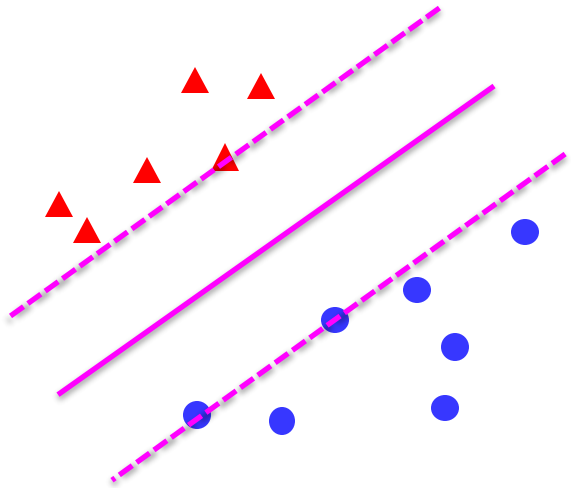
Support Vector Machines

- Convex Optimization Problem:
  - $\min_{w,b} \frac{1}{2}\|w\|^2$
  - subject to:
    - $y_i(w^T x_i + b) \geq 1$ i=1,…,n



Optimization Problem via Duality

- SVM optimization problem:
  - $\max_{\alpha:\ \alpha_i \geq 0} \sum_{i=1}^{n} \alpha_i - \frac{1}{2} \sum_{i=1}^{n} \sum_{j=1}^{n} \alpha_i \alpha_j y_i y_j x_i^T x_j$
  - subject to:
    - $\sum_{i=1}^{n} \alpha_i y_i = 0$

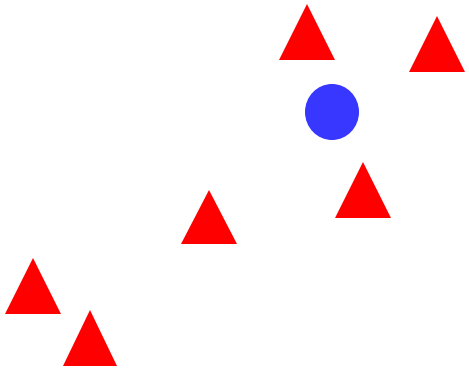


- Once solved, solution hyperplane satisfies:
  - $w = \sum_{i=1}^{n} \alpha_i y_i x_i$
  - $b = -\frac{1}{2}\left(\min_{i:y_i=1} w^T x_i + \max_{i:y_i=-1} w^T x_i\right)$

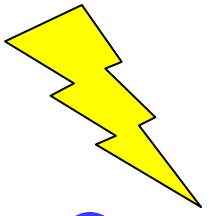
- $\alpha_i \neq 0$ only for the support vectors.

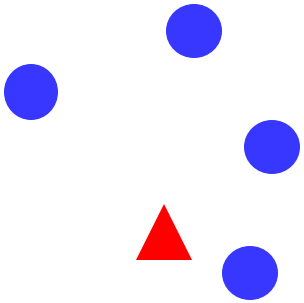
Support Vector Machines

- Max margin separating hyperplane



Not linearly separable



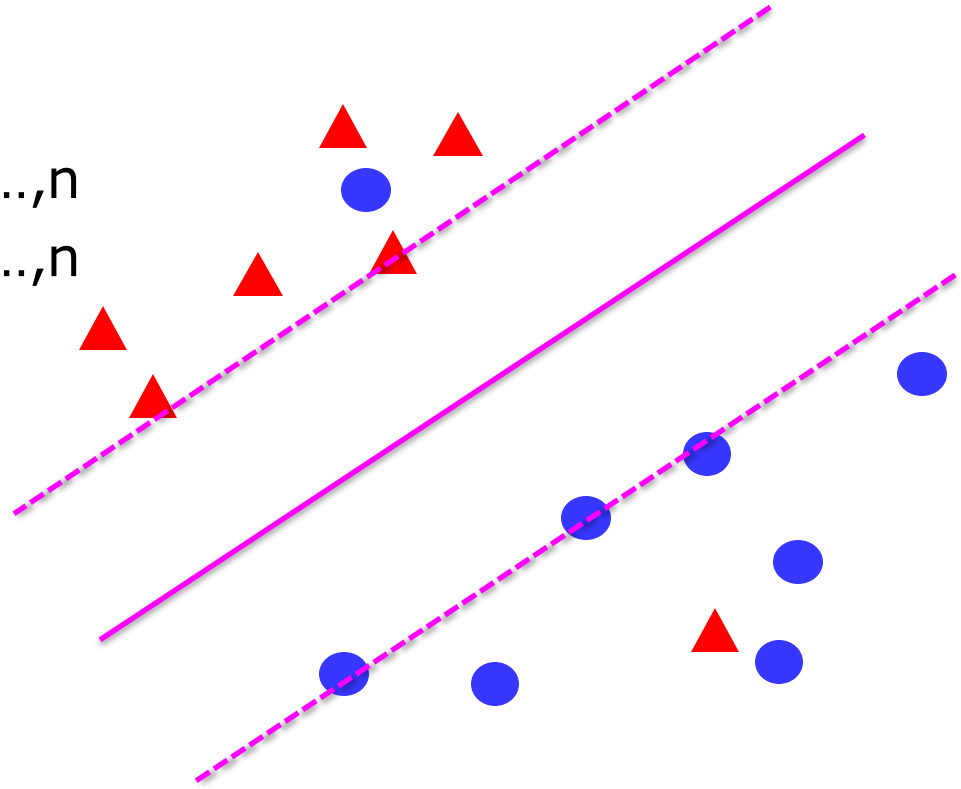


Using complex feature transform to make separable does not seem like right solution

Seems like a few noisy points

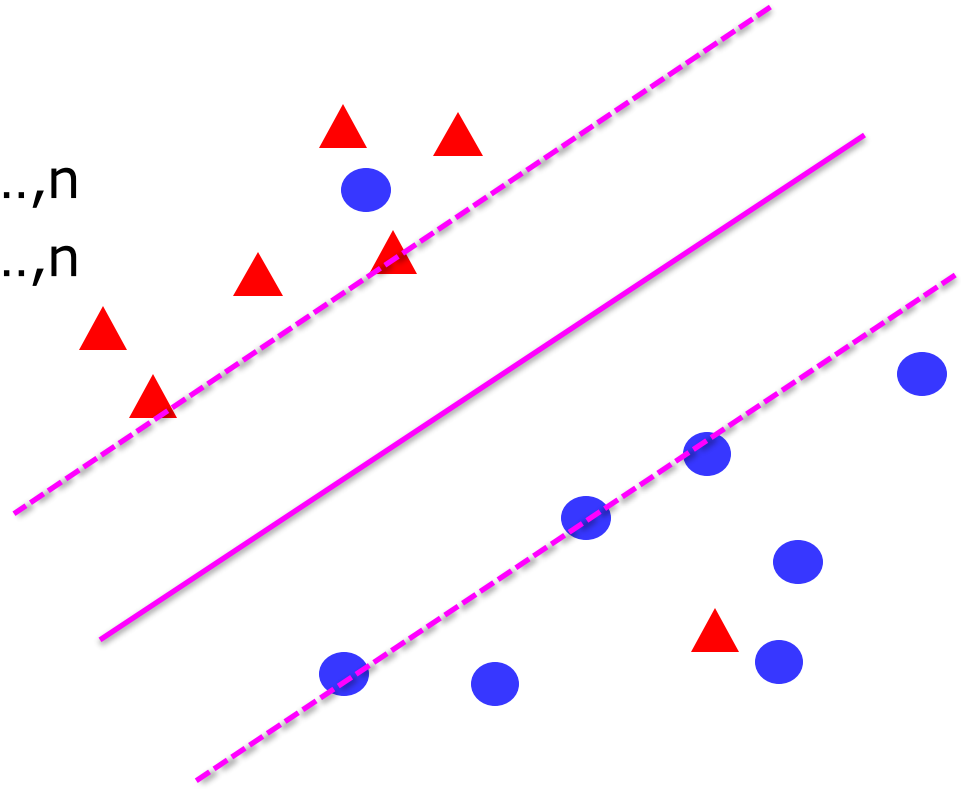
Regularization

- Allow some points to be misclassified or have smaller margin:
  - $\min_{w,b,\xi} \frac{1}{2}\|w\|^2 + C\sum_{i=1}^{n}\xi_i$
  - subject to:
    - $y_i(w^T x_i + b) \geq 1 - \xi_i$ i=1,…,n
    - $\xi_i \geq 0$ i=1,…,n
- Penalty:
  - If functional margin on (x_i, y_i) is ξ_i too small, pay $C\xi_i$.



Regularization

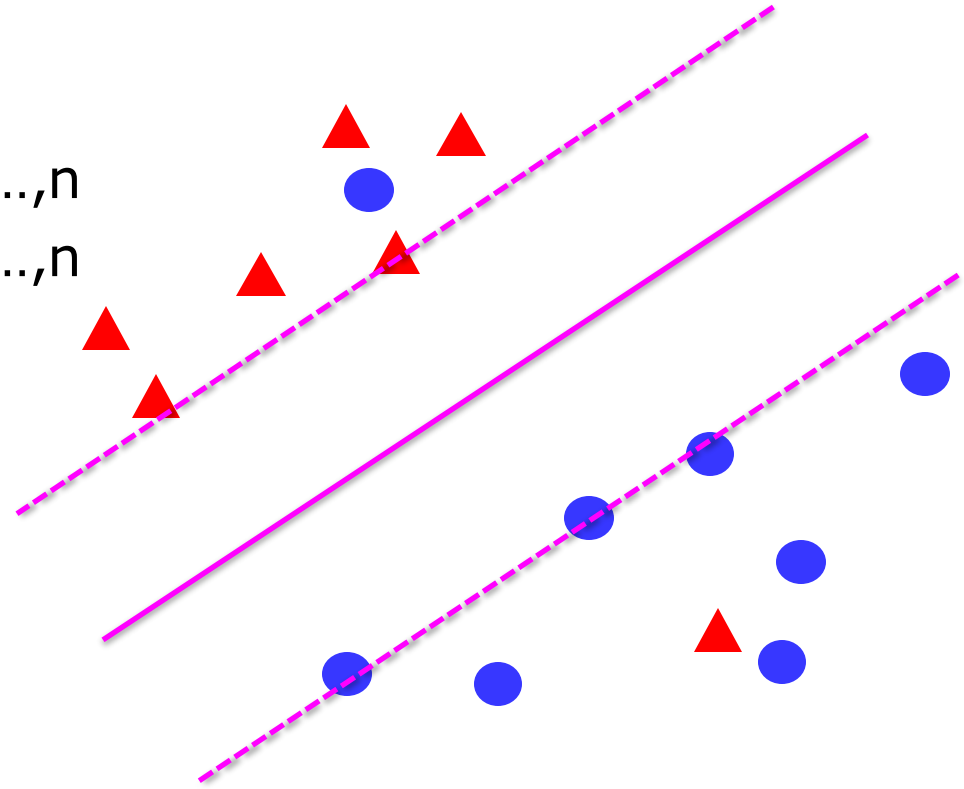
- Allow some points to be misclassified or have smaller margin:
  - $\min_{w,b,\xi} \frac{1}{2}\|w\|^2 + C\sum_{i=1}^{n}\xi_i$
  - subject to:
    - $y_i(w^T x_i + b) \geq 1 - \xi_i$ i=1,…,n
    - $\xi_i \geq 0$ i=1,…,n



- Still Convex:
  - Already showed earlier that linear functions are convex.
    - $C\sum_{i=1}^{n}\xi_i$
    - $\xi_i \geq 0$
  - Both linear.

Regularization

- Regularized SVM:
  - $\min_{w,b,\xi} \frac{1}{2}\|w\|^2 + C\sum_{i=1}^{n}\xi_i$
  - subject to:
    - $y_i(w^T x_i + b) \geq 1 - \xi_i$ i=1,…,n
    - $\xi_i \geq 0$ i=1,…,n



Primal variables

Dual variables

- Dual:
- $\max_{\alpha,r\geq 0} \min_{w,b,\xi} L(w,b,\xi,\alpha,r)$
- $\max_{\alpha,r\geq 0} \min_{w,b,\xi} \frac{1}{2}\|w\|^2 + C\sum_{i=1}^{n}\xi_i - \sum_{i=1}^{n}\alpha_i(y_i(w^T x_i + b) - 1 + \xi_i) - \sum_{i=1}^{n} r_i\xi_i$

Convex Optimization

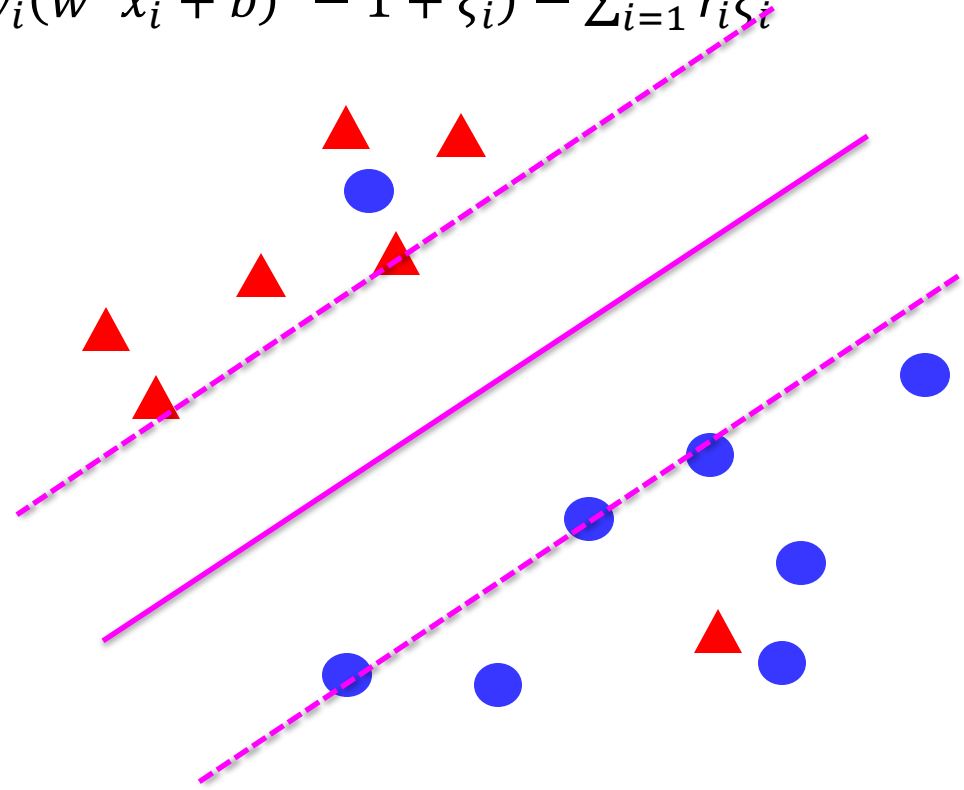
- (KKT) Karush-Kuhn-Tucker conditions:
  - If f and g_i's are convex functions and the constraints g_i are strictly feasible (there is an x with $g_i(x) < 0$ for all i), then
    - $d^* = p^*$
    - There is an optimal solution x*,α* satisfying:
      - $\frac{\partial L(x^*,\alpha^*)}{\partial x_i} = 0$ for i=1,…,d
      - $\alpha_i^* g_i(x^*) = 0$ for i=1,…,n
      - $g_i(x^*) \leq 0$ for i=1,…,n
      - $\alpha_i^* \geq 0$ for i=1,…,n
- Implications:
  - Can solve dual problem, and, assume solution satisfies all of the above.

Regularization

- Dual:
- $\max_{\alpha,r\geq 0} \min_{w,b,\xi} \frac{1}{2}\|w\|^2 + C\sum_{i=1}^{n}\xi_i - \sum_{i=1}^{n}\alpha_i(y_i(w^T x_i + b) - 1 + \xi_i) - \sum_{i=1}^{n} r_i\xi_i$

- Setting $\frac{\partial L(w,b,\xi,\alpha,r)}{\partial w_j} = 0$:
  - $\frac{\partial L(w,b,\xi,\alpha,r)}{\partial w_j} = w_j - \sum_{i=1}^{n}\alpha_i y_i x_{i,j}$.
  - $w = \sum_{i=1}^{n}\alpha_i y_i x_i$

Unchanged

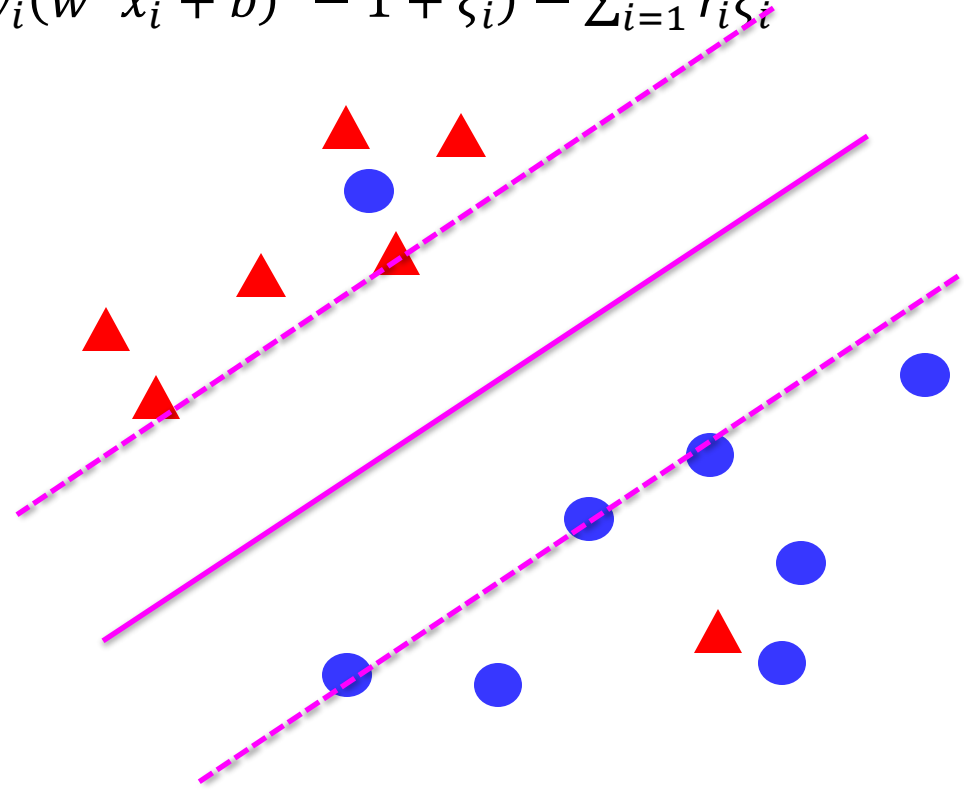


Regularization

- Dual:
- $\max_{\alpha,r\geq 0} \min_{w,b,\xi} \frac{1}{2}\|w\|^2 + C\sum_{i=1}^{n}\xi_i - \sum_{i=1}^{n}\alpha_i(y_i(w^T x_i + b) - 1 + \xi_i) - \sum_{i=1}^{n} r_i\xi_i$

- Setting $\frac{\partial L(w,b,\xi,\alpha,r)}{\partial b} = 0$:
  - $\frac{\partial L(w,b,\xi,\alpha,r)}{\partial b} = -\sum_{i=1}^{n}\alpha_i y_i$
  - $\sum_{i=1}^{n}\alpha_i y_i = 0$

  Unchanged

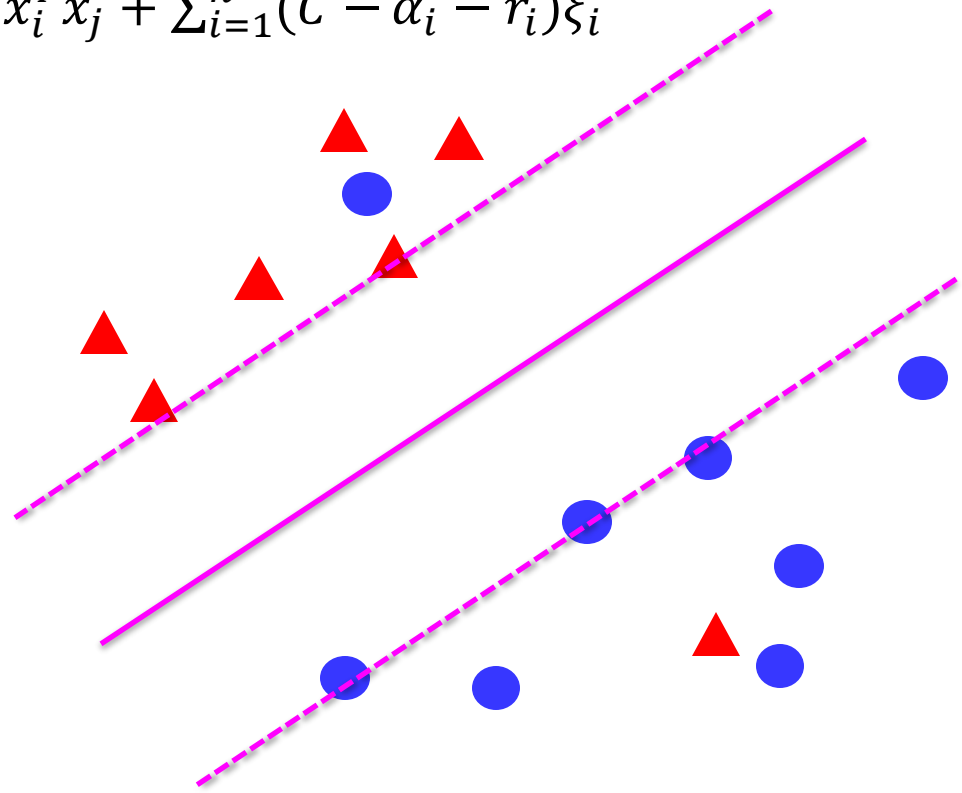


Regularization

- Dual:
- $\max_{\alpha,r\geq 0} \min_{w,b,\xi} \frac{1}{2}\|w\|^2 + C\sum_{i=1}^{n}\xi_i - \sum_{i=1}^{n}\alpha_i(y_i(w^T x_i + b) - 1 + \xi_i) - \sum_{i=1}^{n} r_i\xi_i$
- Becomes (same substitutions as last lecture):
- $\max_{\alpha,r\geq 0} \min_{w,b,\xi} \sum_{i=1}^{n}\alpha_i - \frac{1}{2}\sum_{i=1}^{n}\sum_{j=1}^{n}\alpha_i\alpha_j y_i y_j x_i^T x_j + \sum_{i=1}^{n}(C - \alpha_i - r_i)\xi_i$
- subject to:
  - $\sum_{i=1}^{n}\alpha_i y_i = 0$

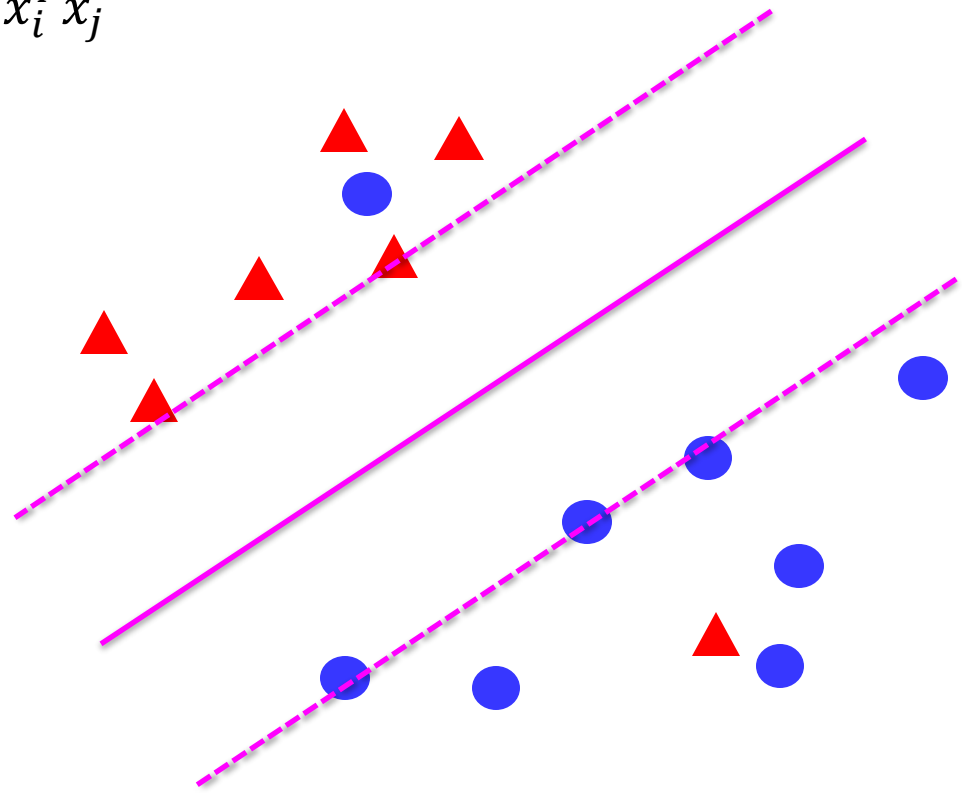
Regularization

- Dual:
- $\max_{\alpha,r\geq 0} \min_{w,b,\xi} \sum_{i=1}^{n} \alpha_i - \frac{1}{2} \sum_{i=1}^{n} \sum_{j=1}^{n} \alpha_i \alpha_j y_i y_j x_i^T x_j + \sum_{i=1}^{n} (C - \alpha_i - r_i) \xi_i$
- subject to:
  - $\sum_{i=1}^{n} \alpha_i y_i = 0$
- Setting $\frac{\partial L(w,b,\xi,\alpha,r)}{\partial \xi_i} = 0$:
  - $\frac{\partial L(w,b,\xi,\alpha,r)}{\partial \xi_i} = C - \alpha_i - r_i$
  - $C = \alpha_i + r_i$



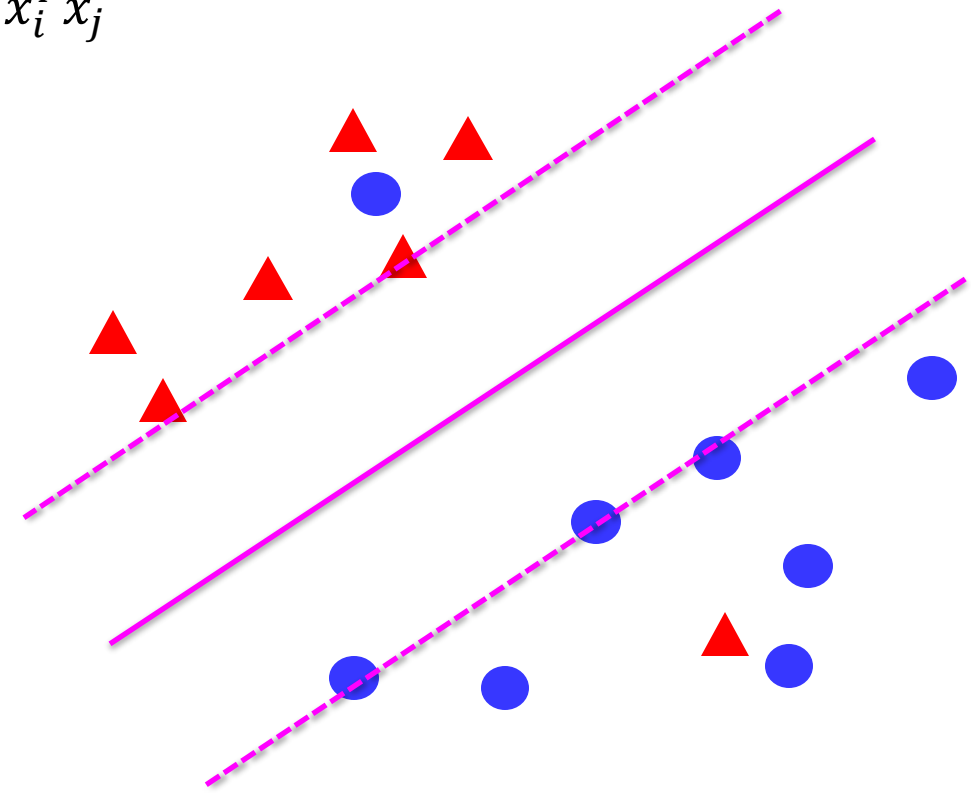
Regularization

- Dual:
- $\max_{\alpha, r \geq 0} \min_{w,b,\xi} \sum_{i=1}^{n} \alpha_i - \frac{1}{2} \sum_{i=1}^{n} \sum_{j=1}^{n} \alpha_i \alpha_j y_i y_j x_i^T x_j$
- subject to:
  - $\sum_{i=1}^{n} \alpha_i y_i = 0$
  - $C = \alpha_i + r_i$ i=1,…,n
- Observation:
  - r does not appear in cost.
  - Can always choose $r_i \geq 0$ to satisfy $C = \alpha_i + r_i$ when $\alpha_i \leq C$.



Regularization

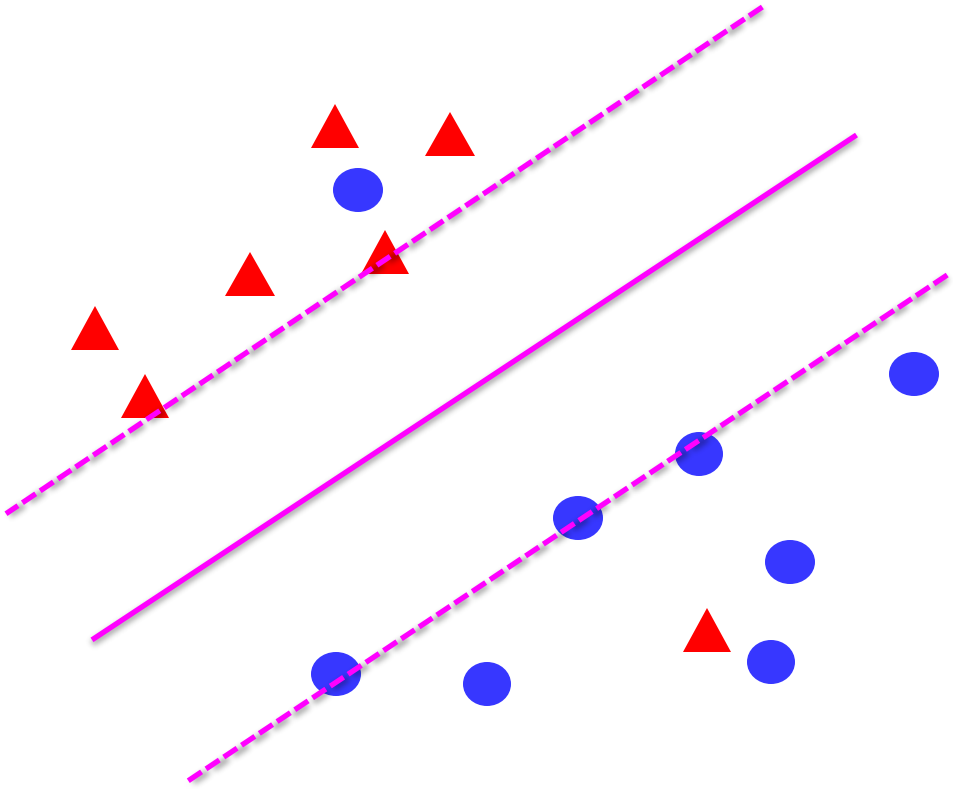
- Dual:
- $\max_{\alpha} \min_{w,b,\xi} \sum_{i=1}^{n} \alpha_i - \frac{1}{2} \sum_{i=1}^{n} \sum_{j=1}^{n} \alpha_i \alpha_j y_i y_j x_i^T x_j$
- subject to:
  - $\sum_{i=1}^{n} \alpha_i y_i = 0$
  - $0 \leq \alpha_i \leq C$ i=1,…,n
- Observation:
  - w, b and ξ do not appear.



Regularization

- Regularized SVM:
- $\max_{\alpha} \sum_{i=1}^{n} \alpha_i - ½ \sum_{i=1}^{n} \sum_{j=1}^{n} \alpha_i \alpha_j y_i y_j x_i^T x_j$
- subject to:
  - $\sum_{i=1}^{n} \alpha_i y_i = 0$
  - $0 \leq \alpha_i \leq C$ i=1,…,n

- Original SVM problem:
- $\max_{\alpha} \sum_{i=1}^{n} \alpha_i - ½ \sum_{i=1}^{n} \sum_{j=1}^{n} \alpha_i \alpha_j y_i y_j x_i^T x_j$
- subject to:
  - $\sum_{i=1}^{n} \alpha_i y_i = 0$
  - $0 \leq \alpha_i$ i=1,…,n

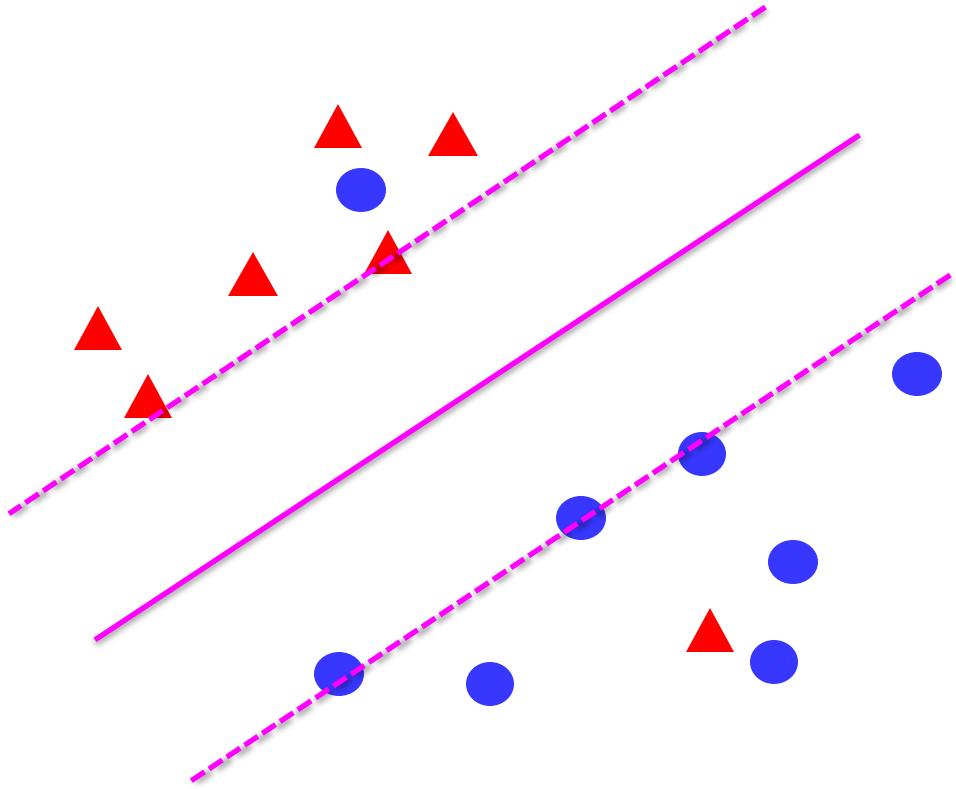


Regularization

- Regularized SVM:
- $\max_{\alpha} \sum_{i=1}^{n} \alpha_i - \frac{1}{2} \sum_{i=1}^{n} \sum_{j=1}^{n} \alpha_i \alpha_j y_i y_j x_i^T x_j$
- subject to:
  - $\sum_{i=1}^{n} \alpha_i y_i = 0$
  - $0 \leq \alpha_i \leq C$ i=1,…,n

- Simply added an upper bound constraint on α_i.
- Still have:

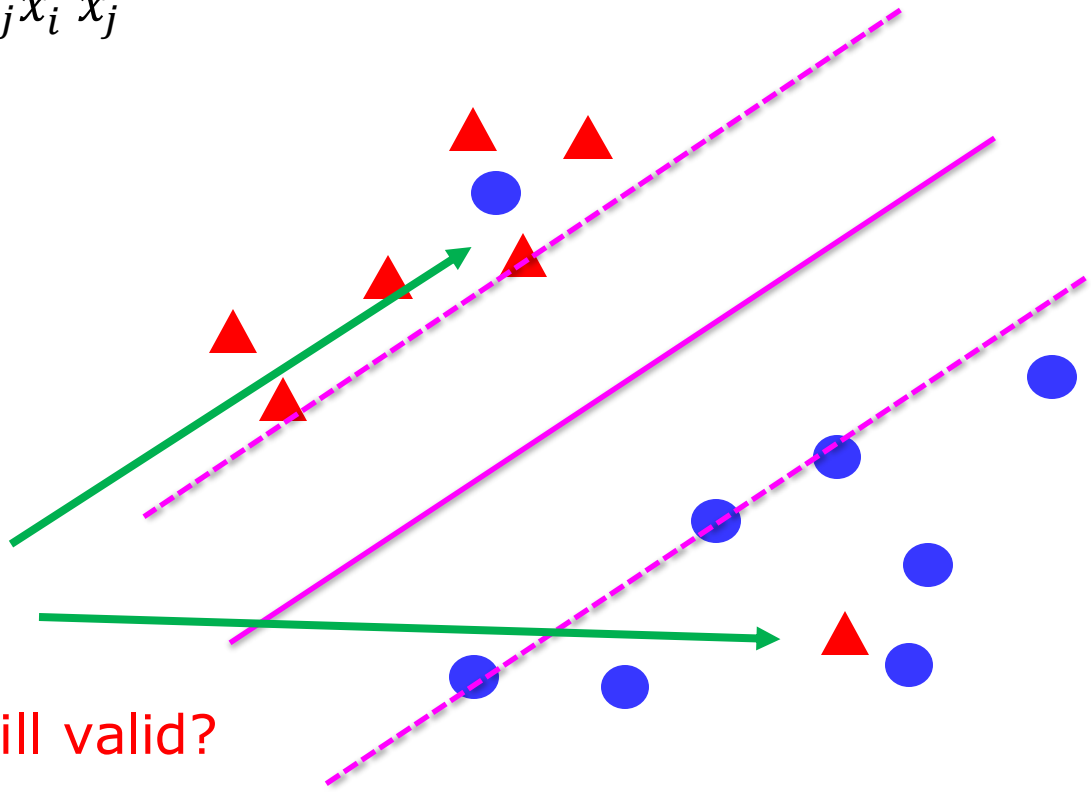
$$w = \sum_{i=1}^{n} \alpha_i y_i x_i .$$



Regularization

- Regularized SVM:
- $\max_{\alpha} \sum_{i=1}^{n} \alpha_i - ½ \sum_{i=1}^{n} \sum_{j=1}^{n} \alpha_i \alpha_j y_i y_j x_i^T x_j$
- subject to:
  - $\sum_{i=1}^{n} \alpha_i y_i = 0$
  - $0 \leq \alpha_i \leq C$ i=1,…,n
- After solving:

$$w = \sum_{i=1}^{n} \alpha_i y_i x_i .$$



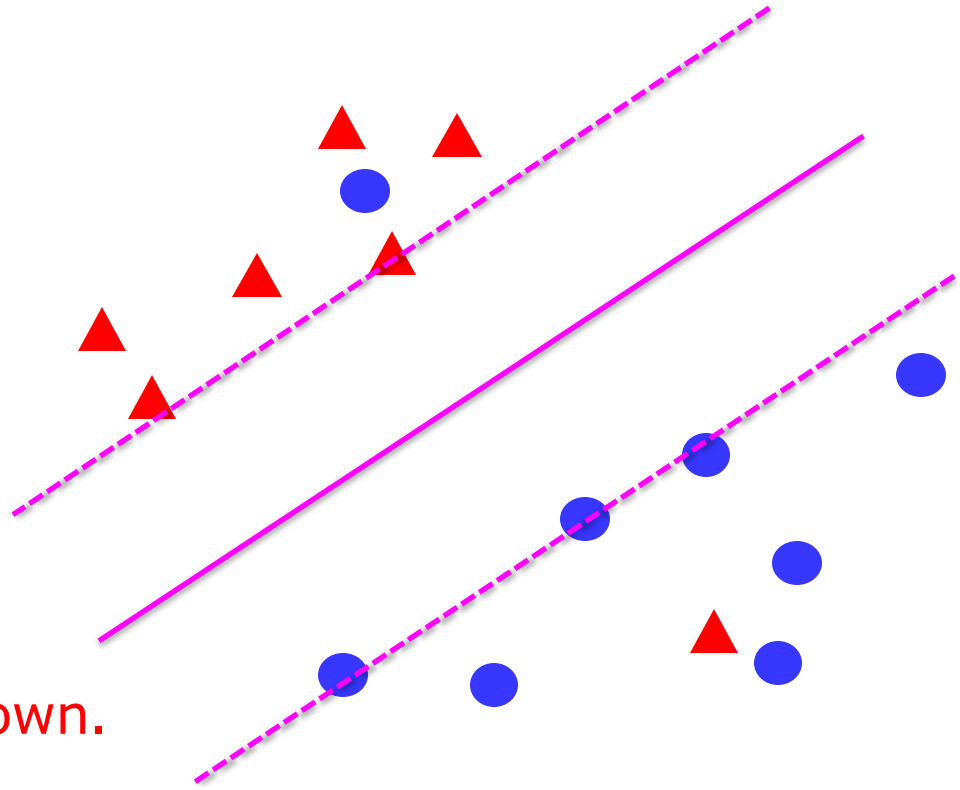
$b = -\frac{1}{2}(\min_{i:y_i=1} w^T x_i + \max_{i:y_i=-1} w^T x_i)$ still valid?

Regularization

- Regularized SVM:
- $\max_{\alpha} \sum_{i=1}^{n} \alpha_i - ½ \sum_{i=1}^{n} \sum_{j=1}^{n} \alpha_i \alpha_j y_i y_j x_i^T x_j$
- subject to:
  - $\sum_{i=1}^{n} \alpha_i y_i = 0$
  - $0 \leq \alpha_i \leq C$ i=1,…,n

- After solving:

$$w = \sum_{i=1}^{n} \alpha_i y_i x_i .$$



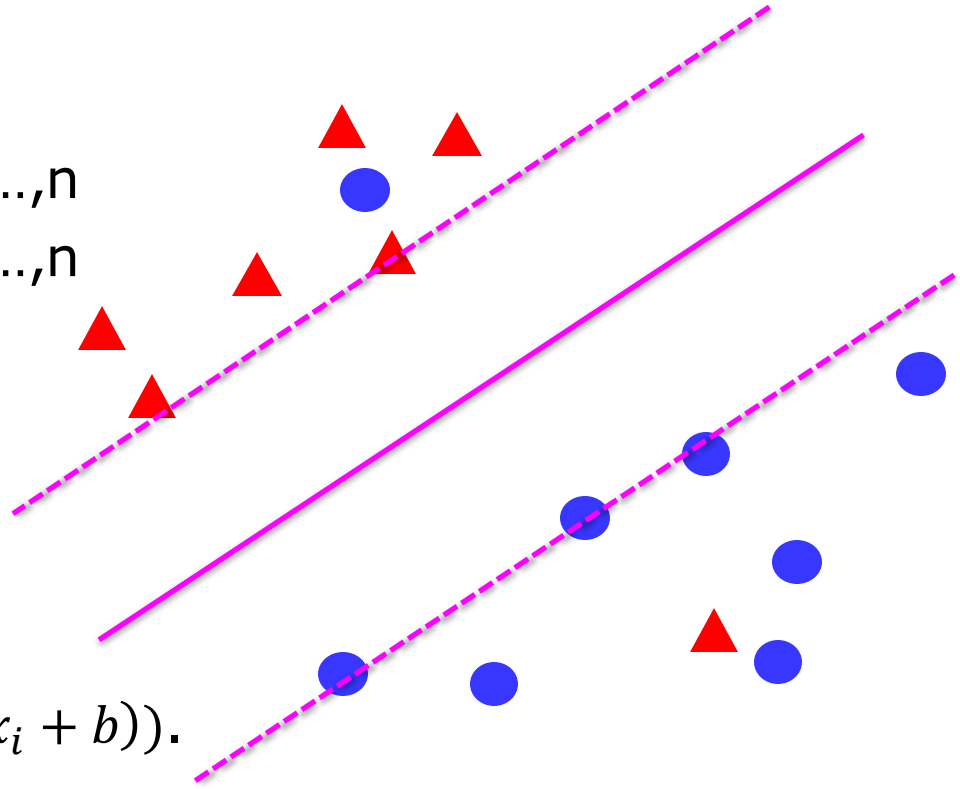
Exercise:

Show how to choose b when w is known.

Everything still works with kernels.

Alternative Approach

- Regularized SVM:
  - $\min_{w,b,\xi} \frac{1}{2}\|w\|^2 + C\sum_{i=1}^{n}\xi_i$
  - subject to:
    - $y_i(w^T x_i + b) \geq 1 - \xi_i$ i=1,…,n
    - $\xi_i \geq 0$ i=1,…,n

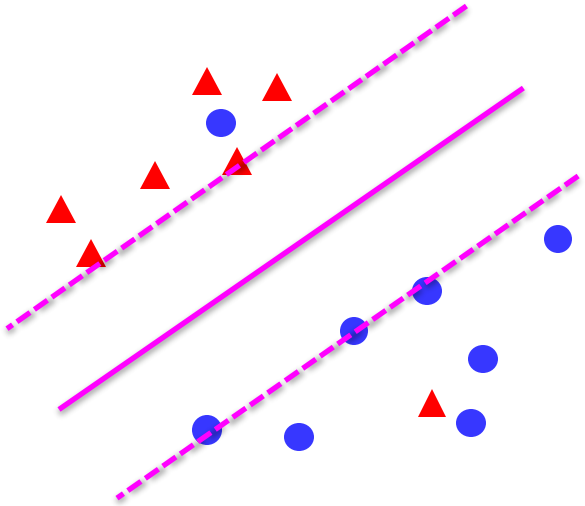


- Unconstrained alternative:

$$\min_{w,b} \frac{1}{2}\|w\|^2 + C\sum_{i=1}^{n}\max(0,1 - y_i(w^T x_i + b)).$$

Hinge loss

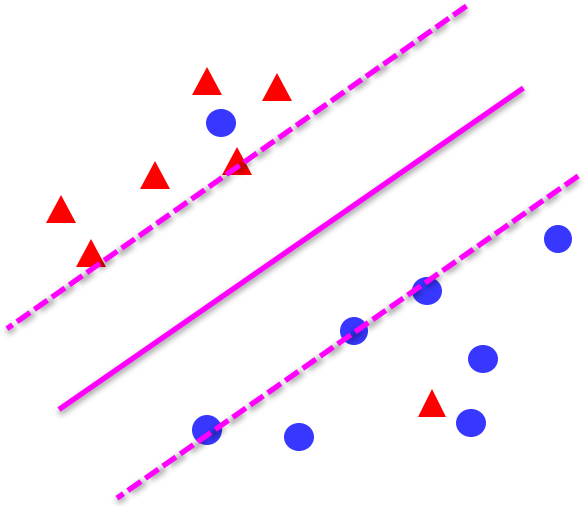
Alternative Approach



- Regularized SVM:
  - $\min_{w,b,\xi} \frac{1}{2}\|w\|^2 + C\sum_{i=1}^{n}\xi_i$
  - subject to:
    - $y_i(w^T x_i + b) \geq 1 - \xi_i$ i=1,…,n
    - $\xi_i \geq 0$ i=1,…,n
- Unconstrained alternative:

  $\min_{w,b} \frac{1}{2}\|w\|^2 + C\sum_{i=1}^{n}\max(0,1 - y_i(w^T x_i + b))$.
- Claim: Same optimal solution w and b.
  - For any w, b let ξ be optimal in top formulation for those w,b.
    - Must have $y_i(w^T x_i + b) = 1 - \xi_i$ or ξ_i=0 o.w. make ξ_i smaller.
  - Therefore $C\sum_{i=1}^{n}\max(0,1 - y_i(w^T x_i + b)) = C\sum_{i=1}^{n}\max(0, \xi_i) = C\sum_{i=1}^{n}\xi_i$
  - So costs the same in unconstrained formulation.

Alternative Approach



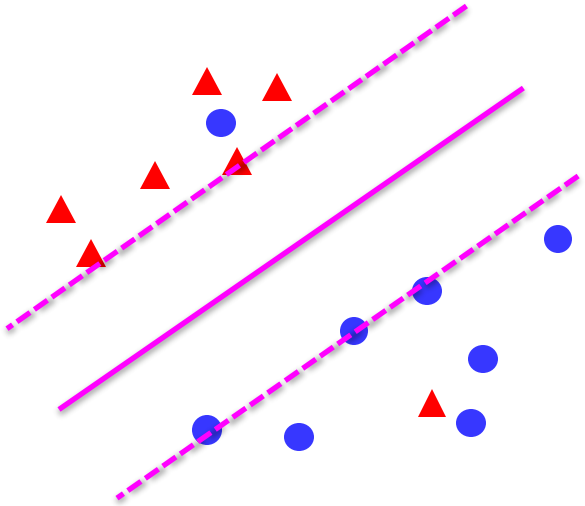
- Regularized SVM:
  - $\min_{w,b,\xi} \frac{1}{2}\|w\|^2 + C\sum_{i=1}^{n}\xi_i$
  - subject to:
    - $y_i(w^T x_i + b) \geq 1 - \xi_i$ i=1,…,n
    - $\xi_i \geq 0$ i=1,…,n
- Unconstrained alternative:

$$\min_{w,b} \frac{1}{2}\|w\|^2 + C\sum_{i=1}^{n}\max(0, 1 - y_i(w^T x_i + b)).$$

- Unconstrained convex optimization problem!
  - Gradient descent on w and b!
  - Easy and efficient!
  - Downside: Does not work with kernels!

What Remains

- Regularized SVM dual:
  - $\max_{\alpha} \sum_{i=1}^{n} \alpha_i - \frac{1}{2} \sum_{i=1}^{n} \sum_{j=1}^{n} \alpha_i \alpha_j y_i y_j x_i^T x_j$
  - subject to:
    - $\sum_{i=1}^{n} \alpha_i y_i = 0$
    - $0 \leq \alpha_i \leq C$ i=1,…,n



- Need to solve above to use with kernels
  - Next: describe ideas in an algorithm for solving it.

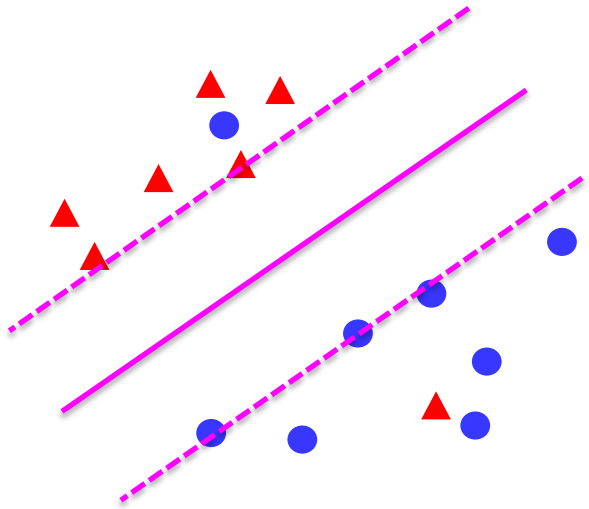
Support Vector Machines

[SVM] 7-9

AARHUS
UNIVERSITY

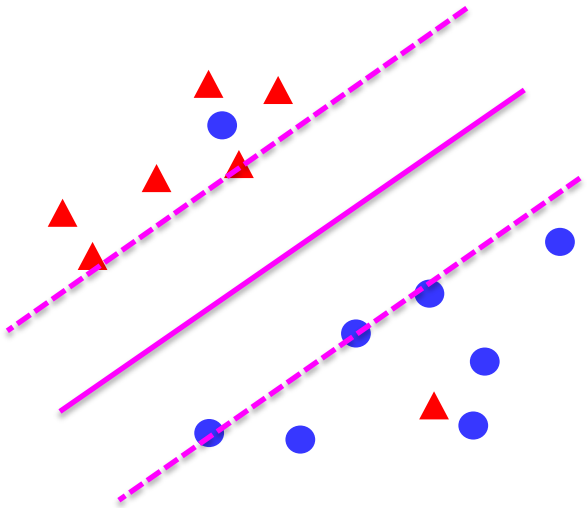
What Remains

- Regularized SVM dual:
  - $\max_{\alpha} \sum_{i=1}^{n} \alpha_i - ½ \sum_{i=1}^{n} \sum_{j=1}^{n} \alpha_i \alpha_j y_i y_j x_i^T x_j$
  - subject to:
    - $\sum_{i=1}^{n} \alpha_i y_i = 0$
    - $0 \leq \alpha_i \leq C$ i=1,…,n



- Need to solve above to use with kernels

Coordinate Ascend



- Regularized SVM dual:
  - $\max_{\alpha} \sum_{i=1}^{n} \alpha_i - \frac{1}{2} \sum_{i=1}^{n} \sum_{j=1}^{n} \alpha_i \alpha_j y_i y_j K(x_i, x_j)$
  - subject to:
    - $\sum_{i=1}^{n} \alpha_i y_i = 0$
    - $0 \leq \alpha_i \leq C$ i=1,…,n

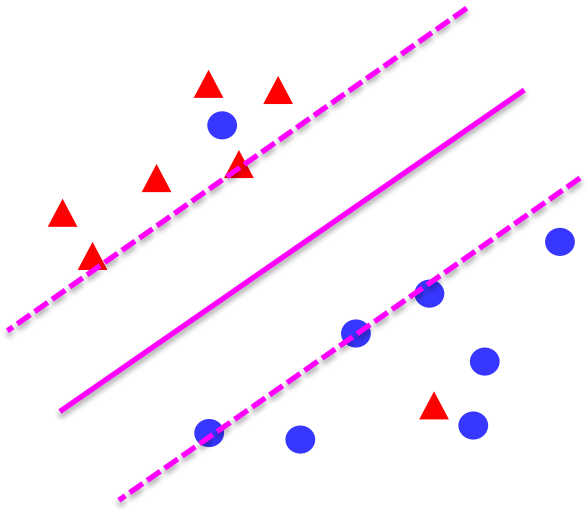
- Coordinate Ascend:
  - Basic idea: Start at feasible α (e.g. all 0).
    - Change α one coordinate at a time.
    - Keep all but α_r fixed and choose α_r to maximize

      $\sum_{i=1}^{n} \alpha_i - \frac{1}{2} \sum_{i=1}^{n} \sum_{j=1}^{n} \alpha_i \alpha_j y_i y_j K(x_i, x_j)$.

      while ensuring all constraints remain satisfied.



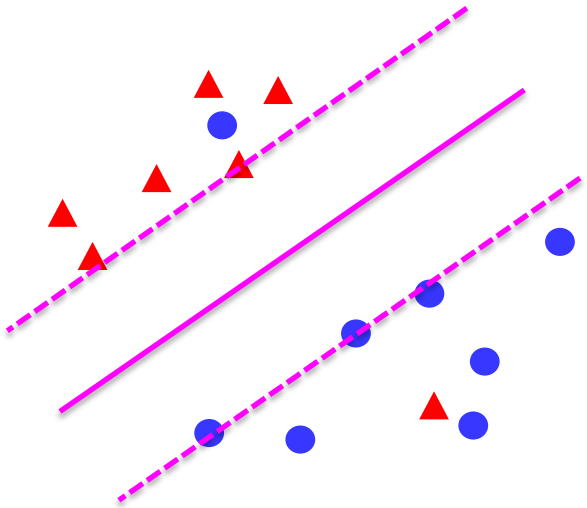
Coordinate Ascend



- Regularized SVM dual:
  - $\max_{\alpha} \sum_{i=1}^{n} \alpha_i - \frac{1}{2} \sum_{i=1}^{n} \sum_{j=1}^{n} \alpha_i \alpha_j y_i y_j K(x_i, x_j)$
  - subject to:
    - $\sum_{i=1}^{n} \alpha_i y_i = 0$
    - $0 \leq \alpha_i \leq C$ i=1,…,n

- Coordinate Ascend:
  - Keep all but α_r fixed and choose α_r to maximize $\sum_{i=1}^{n} \alpha_i - \frac{1}{2} \sum_{i=1}^{n} \sum_{j=1}^{n} \alpha_i \alpha_j y_i y_j K(x_i, x_j)$.
    while ensuring all constraints remain satisfied.
- Issue:
  - $\sum_{i=1}^{n} \alpha_i y_i = 0$ forces $\alpha_r = y_r^{-1} \sum_{i \neq r} \alpha_i y_i$ so cannot change α_r.

Coordinate Ascend

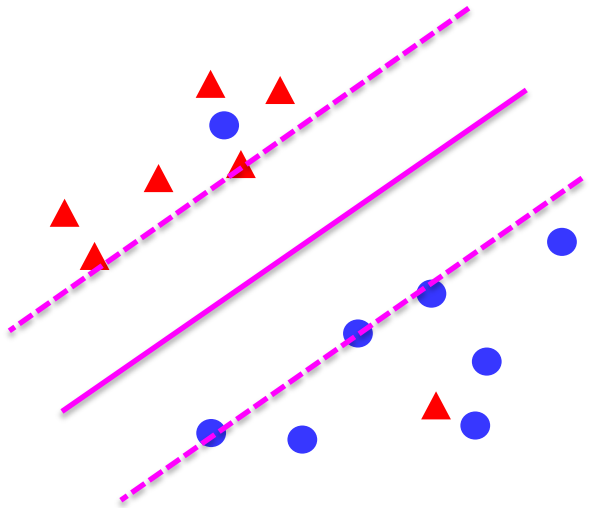


- Regularized SVM dual:
  - $\max_{\alpha} \sum_{i=1}^{n} \alpha_i - \frac{1}{2} \sum_{i=1}^{n} \sum_{j=1}^{n} \alpha_i \alpha_j y_i y_j K(x_i, x_j)$
  - subject to:
    - $\sum_{i=1}^{n} \alpha_i y_i = 0$
    - $0 \leq \alpha_i \leq C$ i=1,…,n

- Ascend on two coordinates:
  - Keep all but α_r, α_s fixed and choose α_r, α_s to maximize $\sum_{i=1}^{n} \alpha_i - \frac{1}{2} \sum_{i=1}^{n} \sum_{j=1}^{n} \alpha_i \alpha_j y_i y_j K(x_i, x_j)$.
    while ensuring all constraints remain satisfied.
- Observation:
  - $\sum_{i=1}^{n} \alpha_i y_i = 0$ forces $\alpha_r y_r + \alpha_s y_s = -\sum_{i \notin \{r,s\}} \alpha_i y_i = \zeta$.
  - So must choose $\alpha_s = y_s^{-1}(\zeta - \alpha_r y_r)$



Coordinate Ascend



- Regularized SVM dual:
  - $\max_{\alpha} \sum_{i=1}^{n} \alpha_i - \frac{1}{2} \sum_{i=1}^{n} \sum_{j=1}^{n} \alpha_i \alpha_j y_i y_j K(x_i, x_j)$
  - subject to:
    - $\sum_{i=1}^{n} \alpha_i y_i = 0$
    - $0 \leq \alpha_i \leq C$ i=1,…,n

- Ascend on two coordinates:
  - Choose α_r to maximize

    $\sum_{i=1}^{n} \alpha_i - \frac{1}{2} \sum_{i=1}^{n} \sum_{j=1}^{n} \alpha_i \alpha_j y_i y_j K(x_i, x_j)$.

    while setting $\alpha_s = y_s^{-1}(\zeta - \alpha_r y_r)$ and ensure all constraints remain satisfied.

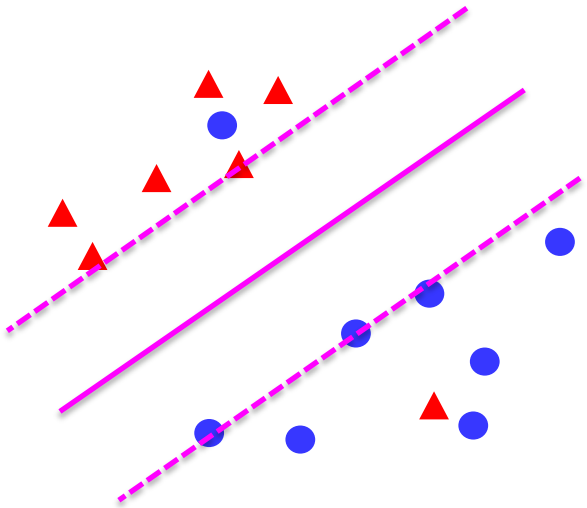
$\zeta = -\sum_{i \notin \{r,s\}} \alpha_i y_i$.

- Observation:
  - Already ensured $\sum_{i=1}^{n} \alpha_i y_i = 0$.



Coordinate Ascend

- Regularized SVM dual:
  - $\max_{\alpha} \sum_{i=1}^{n} \alpha_i - \frac{1}{2} \sum_{i=1}^{n} \sum_{j=1}^{n} \alpha_i \alpha_j y_i y_j K(x_i, x_j)$
  - subject to:
    - $\sum_{i=1}^{n} \alpha_i y_i = 0$
    - $0 \leq \alpha_i \leq C$ i=1,…,n

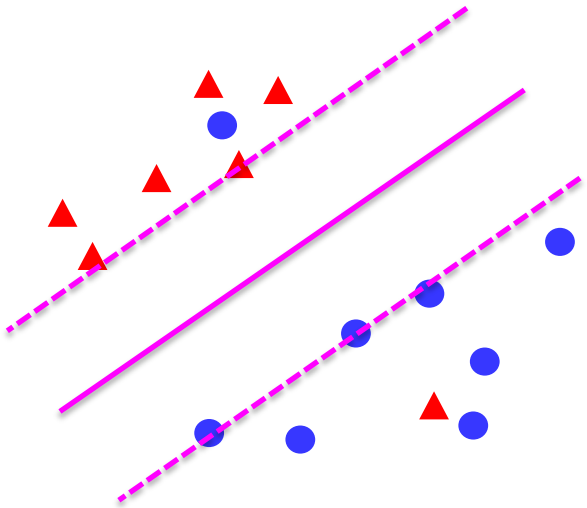


- Ascend on two coordinates:
  - Choose α_r to maximize
    $\sum_{i=1}^{n} \alpha_i - \frac{1}{2} \sum_{i=1}^{n} \sum_{j=1}^{n} \alpha_i \alpha_j y_i y_j K(x_i, x_j)$.
    while setting $\alpha_s = y_s^{-1}(\zeta - \alpha_r y_r)$ and ensure all constraints remain satisfied.

$\zeta = -\sum_{i \notin \{r,s\}} \alpha_i y_i$.

- Observation:
  - Only α_r and α_s change. Must satisfy $0 \leq \alpha_r \leq C$ and $0 \leq \alpha_s \leq C$

Coordinate Ascend



- Regularized SVM dual:
  - $\max_{\alpha} \sum_{i=1}^{n} \alpha_i - \frac{1}{2} \sum_{i=1}^{n} \sum_{j=1}^{n} \alpha_i \alpha_j y_i y_j K(x_i, x_j)$
  - subject to:
    - $\sum_{i=1}^{n} \alpha_i y_i = 0$
    - $0 \leq \alpha_i \leq C$ i=1,…,n

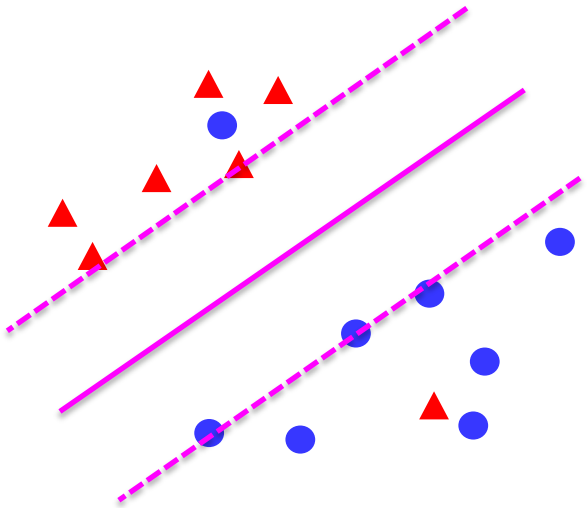
- Ascend on two coordinates:
  - Choose α_r to maximize
    $\sum_{i=1}^{n} \alpha_i - \frac{1}{2} \sum_{i=1}^{n} \sum_{j=1}^{n} \alpha_i \alpha_j y_i y_j K(x_i, x_j)$.
    while setting $\alpha_s = y_s^{-1}(\zeta - \alpha_r y_r)$.

$\zeta = -\sum_{i \notin \{r,s\}} \alpha_i y_i$.

- Observation:
  - Must have $0 \leq y_s^{-1}(\zeta - \alpha_r y_r) \leq C$. Multiply with y_s, subtract ζ, multiply with $-y_r^{-1}$. Inequalities flip whenever multiply with negative. $y_i \in \{-1,1\}$.



Coordinate Ascend



- Regularized SVM dual:
  - $\max_{\alpha} \sum_{i=1}^{n} \alpha_i - \frac{1}{2} \sum_{i=1}^{n} \sum_{j=1}^{n} \alpha_i \alpha_j y_i y_j K(x_i, x_j)$
  - subject to:
    - $\sum_{i=1}^{n} \alpha_i y_i = 0$
    - $0 \leq \alpha_i \leq C$ i=1,…,n

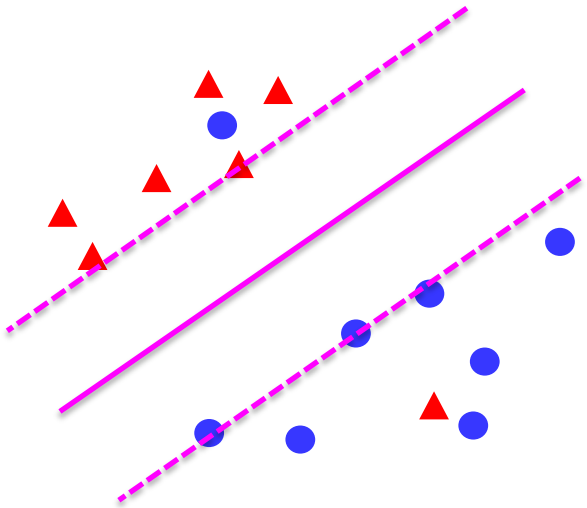
- Ascend on two coordinates:
  - Choose α_r to maximize
    $\sum_{i=1}^{n} \alpha_i - \frac{1}{2} \sum_{i=1}^{n} \sum_{j=1}^{n} \alpha_i \alpha_j y_i y_j K(x_i, x_j)$.
    while setting $\alpha_s = y_s(\zeta - \alpha_r y_r)$.

$\zeta = -\sum_{i \notin \{r,s\}} \alpha_i y_i$.

- Observation:
  - Must have $0 \leq y_s(\zeta - \alpha_r y_r) \leq C$. Multiply with y_s, subtract ζ, multiply with $-y_r$. Inequalities flip whenever multiply with negative. $y_i \in \{-1,1\}$.

Coordinate Ascend

- Regularized SVM dual:
  - $\max_{\alpha} \sum_{i=1}^{n} \alpha_i - \frac{1}{2} \sum_{i=1}^{n} \sum_{j=1}^{n} \alpha_i \alpha_j y_i y_j K(x_i, x_j)$
  - subject to:
    - $\sum_{i=1}^{n} \alpha_i y_i = 0$
    - $0 \leq \alpha_i \leq C$ i=1,…,n



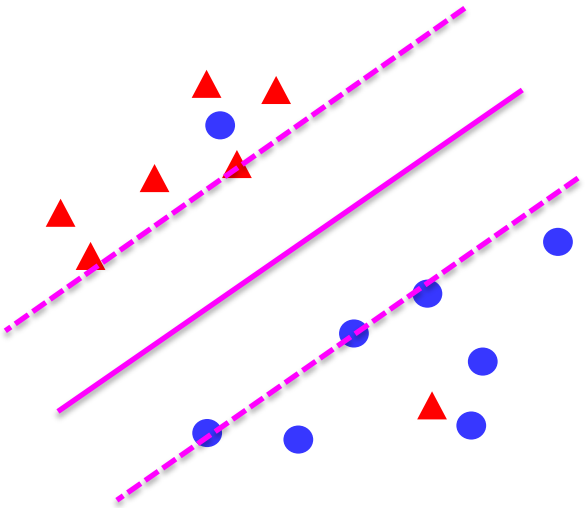
- Ascend on two coordinates:
  - Choose α_r to maximize
    $\sum_{i=1}^{n} \alpha_i - \frac{1}{2} \sum_{i=1}^{n} \sum_{j=1}^{n} \alpha_i \alpha_j y_i y_j K(x_i, x_j)$.
    while setting $\alpha_s = y_s(\zeta - \alpha_r y_r)$.
- If $y_r = y_s$: $-y_r(y_s C - \zeta) \leq \alpha_r \leq y_r \zeta$
- If $y_r \neq y_s$: $y_r \zeta \leq \alpha_r \leq -y_r(y_s C - \zeta)$

$\zeta = -\sum_{i \notin \{r,s\}} \alpha_i y_i$.



Coordinate Ascend

- Regularized SVM dual:
  - $\max_{\alpha} \sum_{i=1}^{n} \alpha_i - \frac{1}{2} \sum_{i=1}^{n} \sum_{j=1}^{n} \alpha_i \alpha_j y_i y_j K(x_i, x_j)$
  - subject to:
    - $\sum_{i=1}^{n} \alpha_i y_i = 0$
    - $0 \leq \alpha_i \leq C$ i=1,…,n

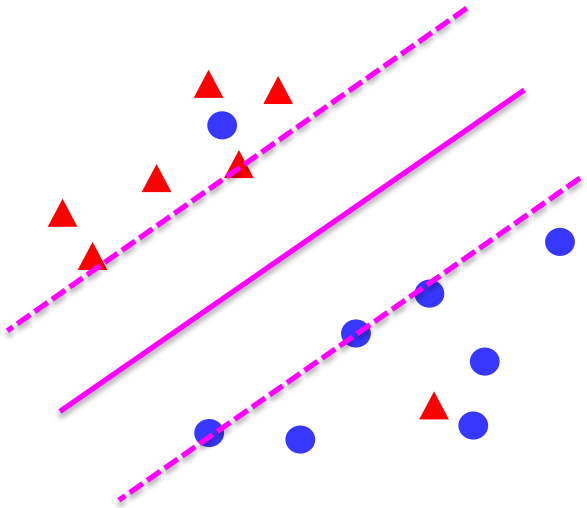


- Ascend on two coordinates:
  - Choose α_r to maximize
    $\sum_{i=1}^{n} \alpha_i - \frac{1}{2} \sum_{i=1}^{n} \sum_{j=1}^{n} \alpha_i \alpha_j y_i y_j K(x_i, x_j)$.
    while setting $\alpha_s = y_s(\zeta - \alpha_r y_r)$.

$\zeta = -\sum_{i \notin \{r,s\}} \alpha_i y_i$.

- If $y_r = y_s$: $\max(0, -y_r(y_s C - \zeta)) \leq \alpha_r \leq \min(C, y_r \zeta)$
- If $y_r \neq y_s$: $\max(0, y_r \zeta) \leq \alpha_r \leq \min(C, -y_r(y_s C - \zeta))$

Coordinate Ascend



- Regularized SVM dual:
  - $\max_{\alpha} \sum_{i=1}^{n} \alpha_i - \frac{1}{2} \sum_{i=1}^{n} \sum_{j=1}^{n} \alpha_i \alpha_j y_i y_j K(x_i, x_j)$
  - subject to:
    - $\sum_{i=1}^{n} \alpha_i y_i = 0$
    - $0 \leq \alpha_i \leq C$ i=1

How to choose α_r?

- Ascend on two coordinates:
  - Choose α_r to maximize

    $\sum_{i=1}^{n} \alpha_i - \frac{1}{2} \sum_{i=1}^{n} \sum_{j=1}^{n} \alpha_i \alpha_j y_i y_j K(x_i, x_j)$.

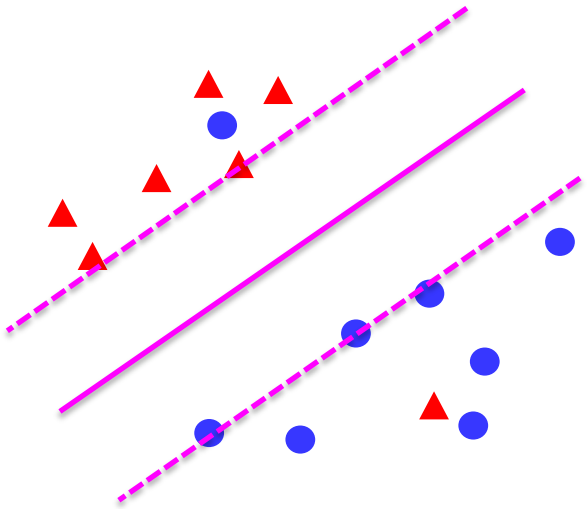
    while setting $\alpha_s = y_s(\zeta - \alpha_r y_r)$.

$\zeta = -\sum_{i \notin \{r,s\}} \alpha_i y_i$.

- If $y_r = y_s$: $\max(0, -y_r(y_s C - \zeta)) \leq \alpha_r \leq \min(C, y_r \zeta)$
- If $y_r \neq y_s$: $\max(0, y_r \zeta) \leq \alpha_r \leq \min(C, -y_r(y_s C - \zeta))$



Coordinate Ascend



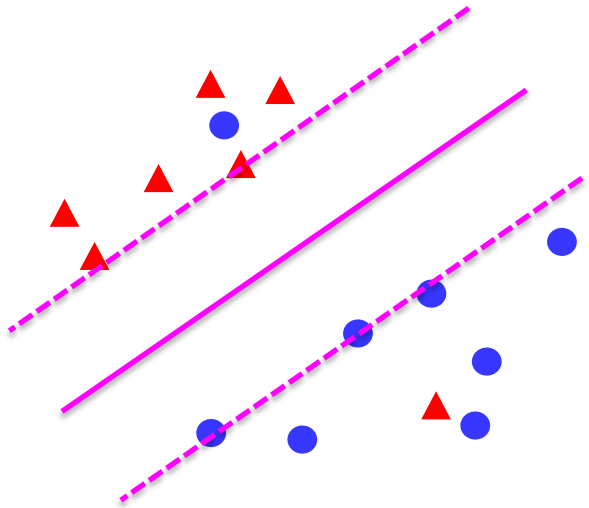
- Regularized SVM dual:
  - $\max_{\alpha} \sum_{i=1}^{n} \alpha_i - \frac{1}{2} \sum_{i=1}^{n} \sum_{j=1}^{n} \alpha_i \alpha_j y_i y_j K(x_i, x_j)$
  - subject to:
    - $\sum_{i=1}^{n} \alpha_i y_i = 0$
    - $0 \leq \alpha_i \leq C$ i=1,…,n

- Ascend on two coordinates:
  - Choose α_r to maximize
    $\sum_{i=1}^{n} \alpha_i - \frac{1}{2} \sum_{i=1}^{n} \sum_{j=1}^{n} \alpha_i \alpha_j y_i y_j K(x_i, x_j)$.
    while setting $\alpha_s = y_s(\zeta - \alpha_r y_r)$.
- Observation: Replace α_s by $y_s(\zeta - \alpha_r y_r)$ in
  $$\sum_{i=1}^{n} \alpha_i - \frac{1}{2} \sum_{i=1}^{n} \sum_{j=1}^{n} \alpha_i \alpha_j y_i y_j K(x_i, x_j)$$
  - When all others are fixed, quadratic function of α_r

$\zeta = -\sum_{i \notin \{r,s\}} \alpha_i y_i$.



Coordinate Ascend



- Regularized SVM dual:
  - $\max_{\alpha} \sum_{i=1}^{n} \alpha_i - \frac{1}{2} \sum_{i=1}^{n} \sum_{j=1}^{n} \alpha_i \alpha_j y_i y_j K(x_i, x_j)$
  - subject to:
    - $\sum_{i=1}^{n} \alpha_i y_i = 0$
    - $0 \leq \alpha_i \leq C$ i=1,…,n

- Ascend on two coordinates:
  - Choose α_r to maximize
    $\sum_{i=1}^{n} \alpha_i - \frac{1}{2} \sum_{i=1}^{n} \sum_{j=1}^{n} \alpha_i \alpha_j y_i y_j K(x_i, x_j)$.
    while setting $\alpha_s = y_s(\zeta - \alpha_r y_r)$.

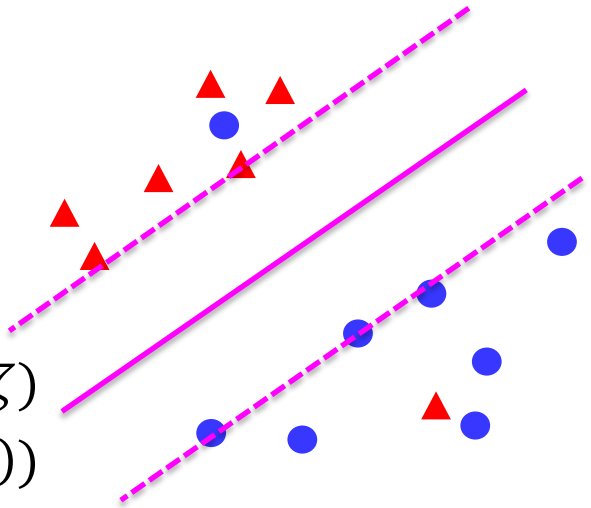
$\zeta = -\sum_{i \notin \{r,s\}} \alpha_i y_i$.

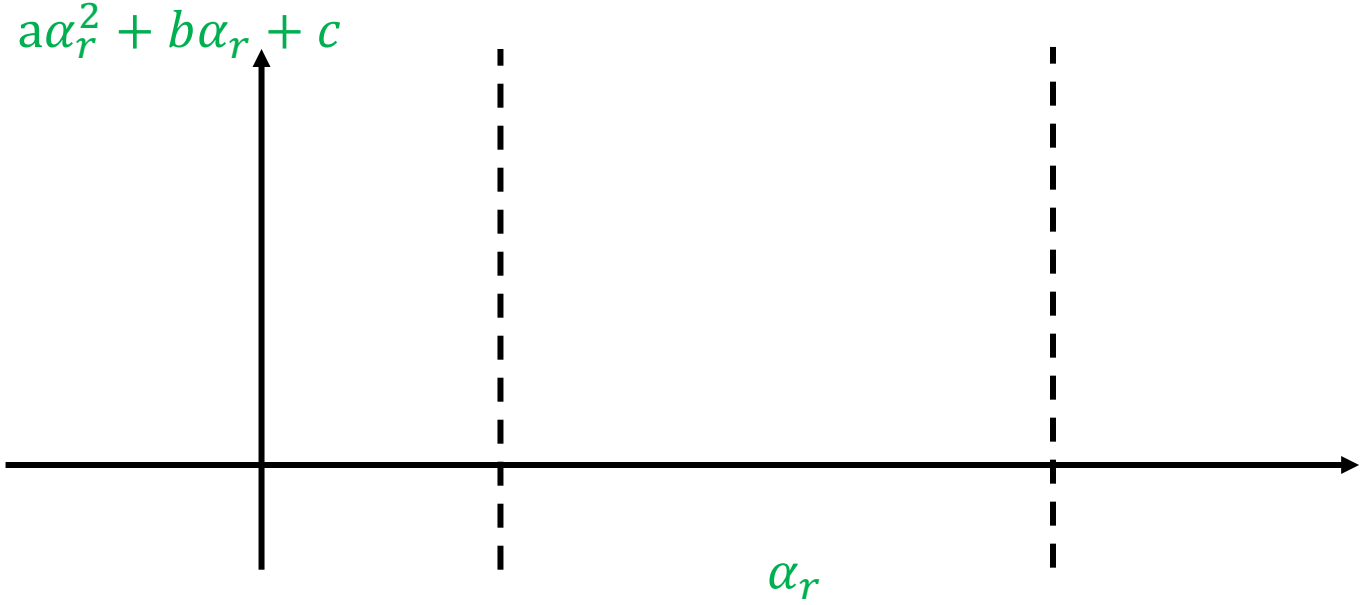
- Observation: Just need to choose α_r to maximize
  - $a\alpha_r^2 + b\alpha_r + c$
  - For some a,b,c and with a constraint on range of α_r.



Coordinate Ascend

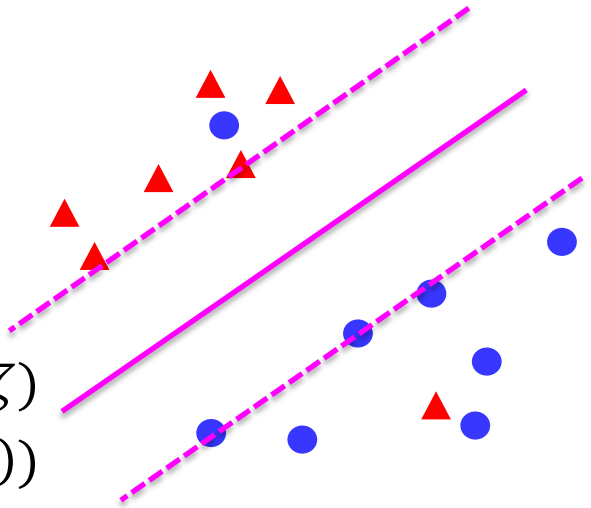
- Task: Choose α_r to maximize
  - $a\alpha_r^2 + b\alpha_r + c$
- For some a,b,c with constraints:
- If $y_r = y_s$: $\max(0, -y_r(y_s C - \zeta)) \leq \alpha_r \leq \min(C, y_r\zeta)$
- If $y_r \neq y_s$: $\max(0, y_r\zeta) \leq \alpha_r \leq \min(C, -y_r(y_s C - \zeta))$

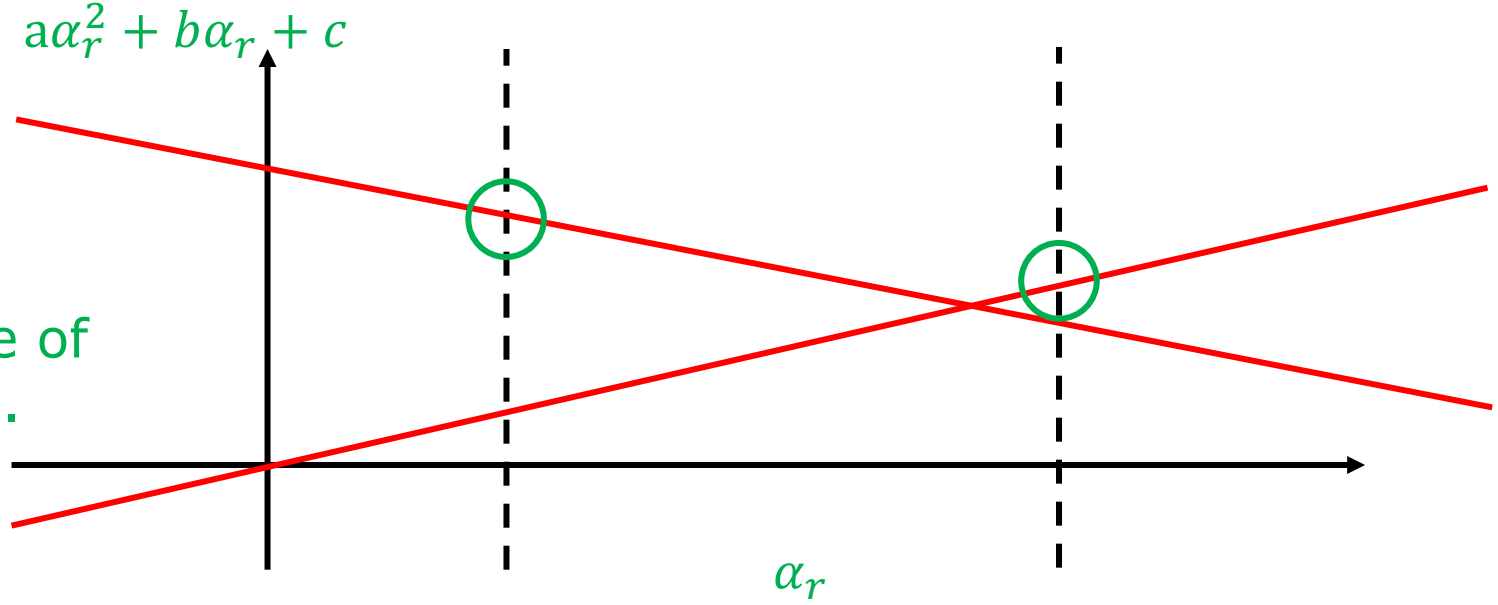




Coordinate Ascend

- Task: Choose α_r to maximize
  - $a\alpha_r^2 + b\alpha_r + c$
- For some a,b,c with constraints:
- If $y_r = y_s$: $\max(0, -y_r(y_s C - \zeta)) \leq \alpha_r \leq \min(C, y_r\zeta)$
- If $y_r \neq y_s$: $\max(0, y_r\zeta) \leq \alpha_r \leq \min(C, -y_r(y_s C - \zeta))$



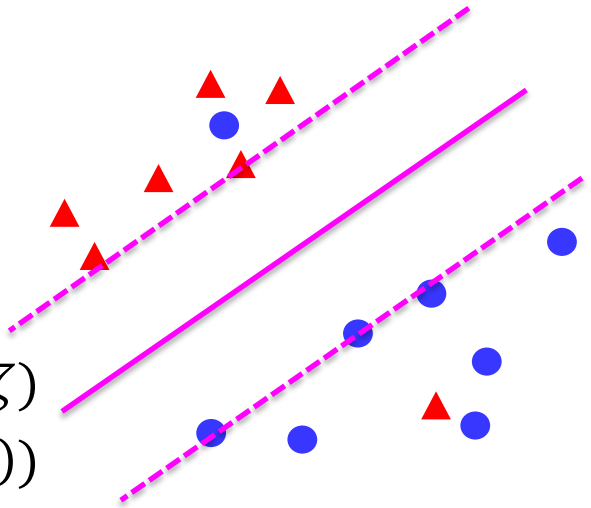


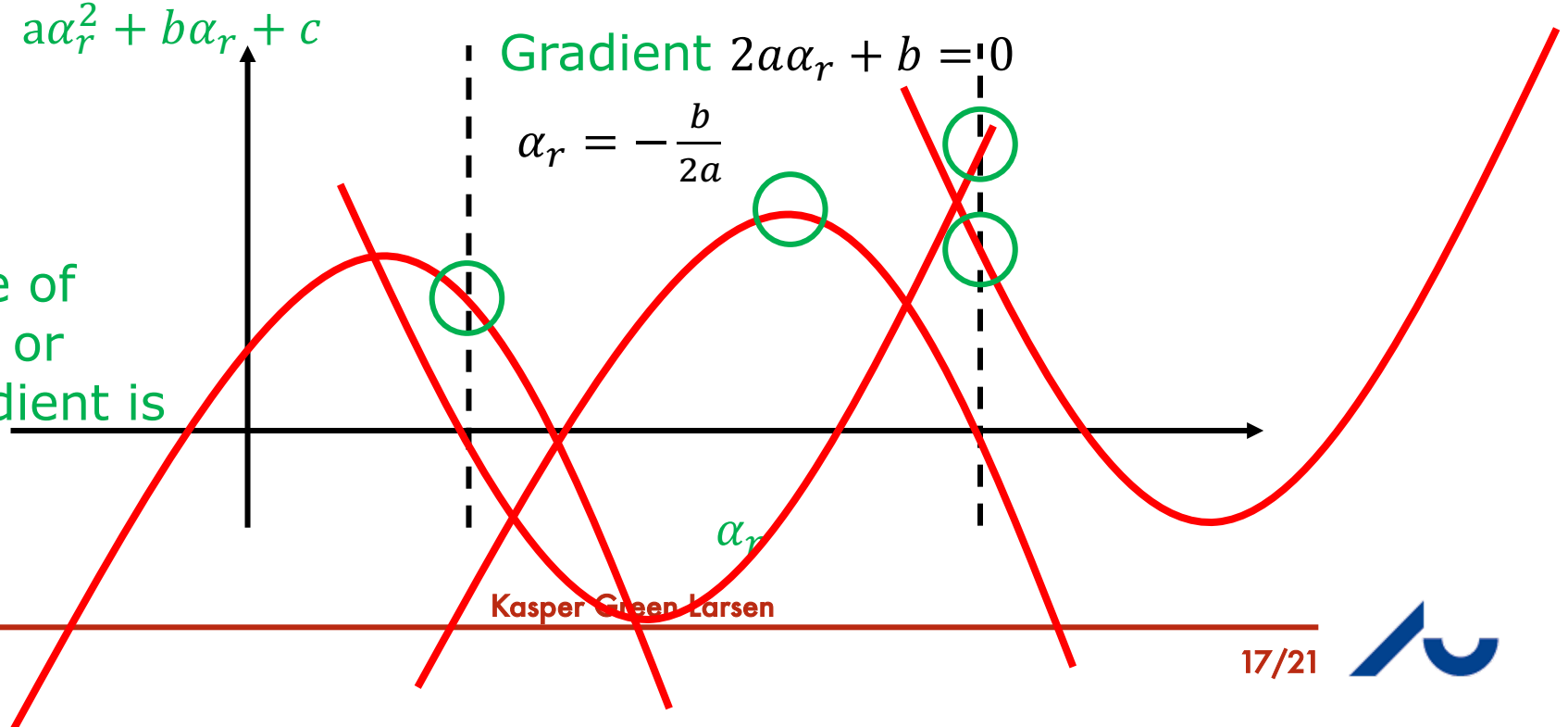
Case a=0

Max at one of end points.

Coordinate Ascend

- Task: Choose α_r to maximize
  - $a\alpha_r^2 + b\alpha_r + c$
- For some a,b,c with constraints:
- If $y_r = y_s$: $\max(0, -y_r(y_s C - \zeta)) \leq \alpha_r \leq \min(C, y_r\zeta)$
- If $y_r \neq y_s$: $\max(0, y_r\zeta) \leq \alpha_r \leq \min(C, -y_r(y_s C - \zeta))$

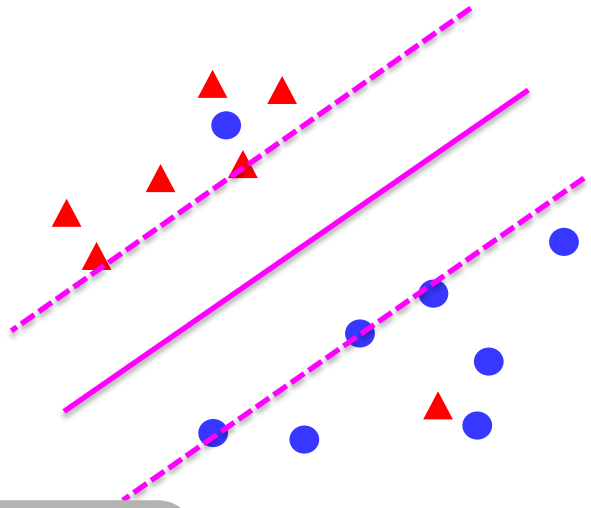




Case a≠0

Max at one of end points or where gradient is 0.

Coordinate Ascend



- Regularized SVM dual:
  - $\max_{\alpha} \sum_{i=1}^{n} \alpha_i - \frac{1}{2} \sum_{i=1}^{n} \sum_{j=1}^{n} \alpha_i \alpha_j y_i y_j K(x_i, x_j)$
  - subject to:
    - $\sum_{i=1}^{n} \alpha_i y_i = 0$
    - $0 \leq \alpha_i \leq C$ i=1,…,n

When to stop?

- SMO Algorithm
  - Start at feasible α
  - Repeat until satisfied:
    - Pick two coordinates α_r, α_s, fix all others and maximize

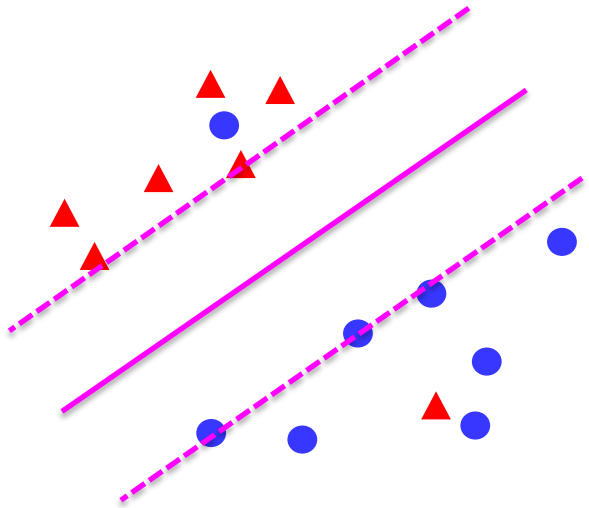
      $\sum_{i=1}^{n} \alpha_i - \frac{1}{2} \sum_{i=1}^{n} \sum_{j=1}^{n} \alpha_i \alpha_j y_i y_j K(x_i, x_j)$.

      while ensuring all constraints remain satisfied. Do so by solving a quadratic equation in α_r.



SMO Algorithm

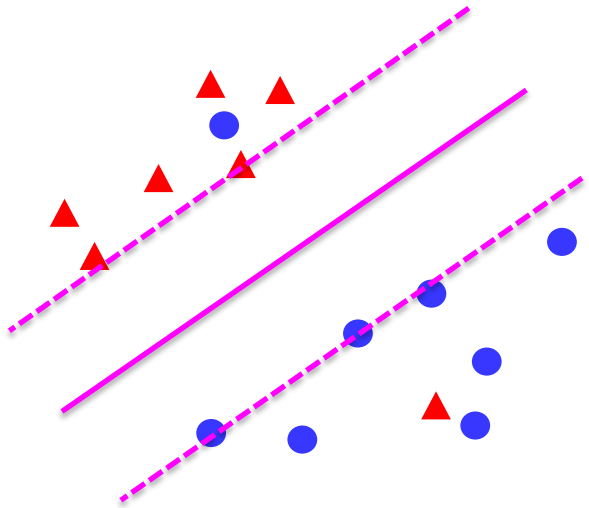
- Based on KKT conditions, stop when:
  - $\alpha_i = 0 \Longrightarrow y_i(w^T x_i + b) \geq 1$
  - $\alpha_i = C \Longrightarrow y_i(w^T x_i + b) \leq 1$
  - $0 < \alpha_i < C \Longrightarrow y_i(w^T x_i + b) = 1$



- Allow a bit of slack! Tolerance ε equal to e.g. 0.01 or 0.001. Stop when:
  - $\alpha_i = 0 \Longrightarrow y_i(w^T x_i + b) \geq 1 - \varepsilon$
  - $\alpha_i = C \Longrightarrow y_i(w^T x_i + b) \leq 1 + \varepsilon$
  - $0 < \alpha_i < C \Longrightarrow y_i(w^T x_i + b) \in (1 - \varepsilon, 1 + \varepsilon)$



SMO Algorithm



- Tolerance ε
  - $\alpha_i = 0 \Rightarrow y_i(w^T x_i + b) \geq 1 - \varepsilon$
  - $\alpha_i = C \Rightarrow y_i(w^T x_i + b) \leq 1 + \varepsilon$
  - $0 < \alpha_i < C \Rightarrow y_i(w^T x_i + b) \in (1 - \varepsilon, 1 + \varepsilon)$

- SMO Algorithm
  - Start at feasible α
  - Repeat until satisfy above constraints:
    - Pick two coordinates α_r, α_s, fix all others and maximize

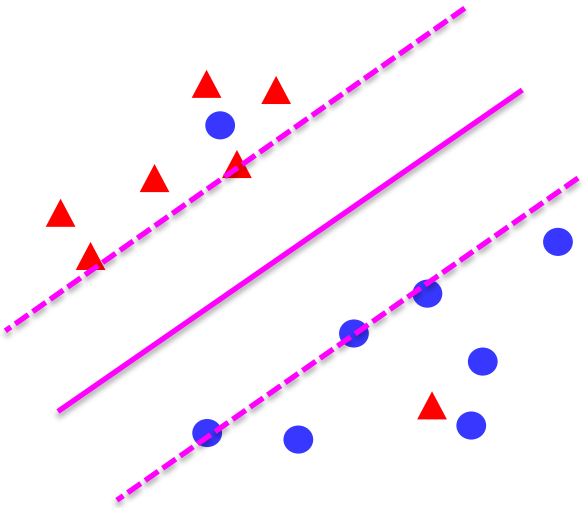
      $\sum_{i=1}^{n} \alpha_i - \frac{1}{2} \sum_{i=1}^{n} \sum_{j=1}^{n} \alpha_i \alpha_j y_i y_j K(x_i, x_j)$.

      while ensuring all constraints remain satisfied. Do so by solving a quadratic equation in α_r.

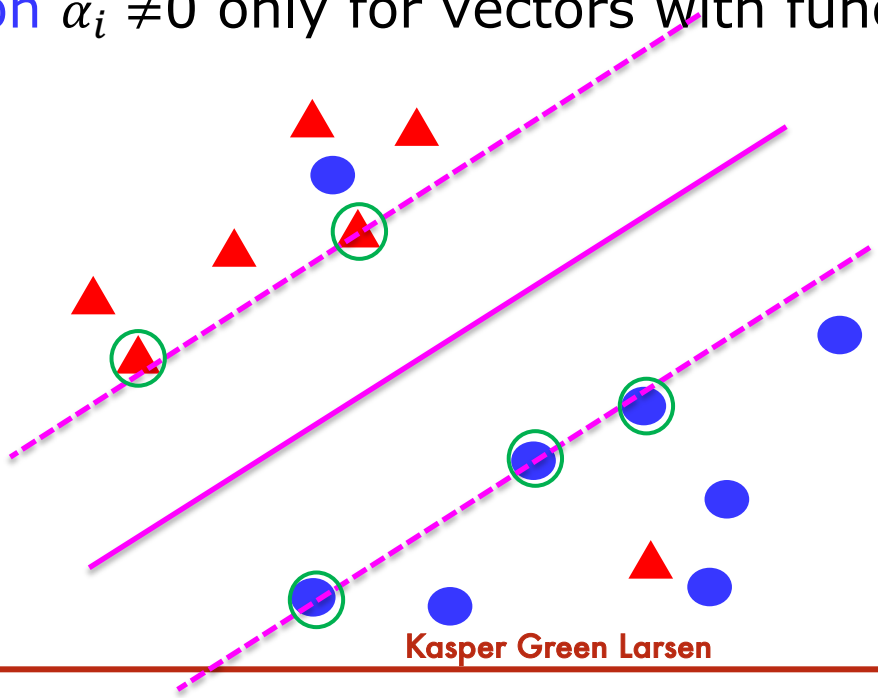


SMO Algorithm

- Tolerance ε
  - $\alpha_i = 0 \Rightarrow y_i(w^T x_i + b) \geq 1 - \varepsilon$
  - $\alpha_i = C \Rightarrow y_i(w^T x_i + b) \leq 1 + \varepsilon$
  - $0 < \alpha_i < C \Rightarrow y_i(w^T x_i + b) \in (1 - \varepsilon, 1 + \varepsilon)$



- Observation $\alpha_i \neq 0$ only for vectors with functional margin $\leq 1 + \varepsilon$



Can still have few "support" vectors.